

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Волгоградский государственный технический университет»

Камышинский технологический институт

(филиал)

федерального государственного бюджетного образовательного учреждения
высшего образования

«Волгоградский государственный технический университет»

ФАКУЛЬТЕТ СРЕДНЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ

КАФЕДРА «АВТОМАТИЗИРОВАННЫЕ СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И
УПРАВЛЕНИЯ»

УТВЕРЖДАЮ

Заведующий кафедрой АСОИУ

 И.М. Харитонов

« 4 » 06 2025 г.

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА К ДИПЛОМНОМУ ПРОЕКТУ СПЕЦИАЛИСТА НА ТЕМУ

Разработка мессенджера «GameChat»

Код дипломного проекта ДП 09.02.07.10.Д.ПЗ
(обозначение документа)

Автор  11.06.2025
(подпись и дата подписания)

Костюк Илья Вадимович
(фамилия, имя, отчество)

Группа КИПС-211
(шифр группы)

Специальность 09.02.07 Информационные системы и программирование
(код, наименование)

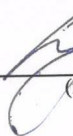
Руководитель проекта  11.06.25
(подпись и дата подписания)

А.Д. Дегтярёв
(инициалы и фамилия)

Консультанты по разделам:
экономическая часть
(краткое наименование раздела)

 11.06.25
(подпись и дата подписания)

Т.В. Бородина
(инициалы и фамилия)

Нормоконтролер  11.06.2025
(подпись и дата подписания)

А.И. Ромашенко
(инициалы и фамилия)

Камышин 2025 г.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Волгоградский государственный технический университет»

Камышинский технологический институт

(филиал)

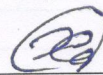
федерального государственного бюджетного образовательного учреждения
высшего образования

«Волгоградский государственный технический университет»

КАФЕДРА «АВТОМАТИЗИРОВАННЫЕ СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И
УПРАВЛЕНИЯ»

УТВЕРЖДАЮ

Заведующий кафедрой АСОИУ



И.М. Харитонов

« 19 » 12 2024 г.

ЗАДАНИЕ

на дипломный проект

Студент

Костюк Илья Вадимович

(фамилия, имя, отчество)

Код кафедры 09.1

Группа КИПС-211

Тема Разработка мессенджера «GameChat»

утверждена приказом по институту от 19.12.2024 № 145-ст

Срок представления готового проекта 11 июня 2025 г.

(дата)



(подпись)

Исходные данные для выполнения работ

1) Гриффитс И. Програмируем на C# 8.0. Разработка приложений. – СПб.: Питер, 2021. – 944 с.

2) Холленд Р. Microsoft SQL Server 2019. Полное руководство. – М.: ДМК Пресс, 2020. – 1400 с.

3) Снеговский М. Построение распределенных систем на основе WCF. – СПб.: Питер, 2017. – 464 с.

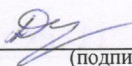
Содержание основной части пояснительной записки

- 1) Постановка целей и задач
- 2) Обзор аналогов
- 3) Выбор инструментов реализации
- 4) Процесс разработки
- 5) Экономическая часть

Перечень графического материала

- 1) Схема клиент-серверной архитектуры
- 2) ER-диаграмма базы данных
- 3) Интерфейс главной формы

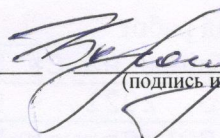
Руководитель проекта

 19.12.24
(подпись и дата подписания)

А.Д. Дегтярёв
(инициалы и фамилия)

Консультанты по разделам
экономическая часть

(краткое наименование раздела)

 19.12.24
(подпись и дата подписания)

Т.В. Бородина
(инициалы и фамилия)

(краткое наименование раздела)

(подпись и дата подписания)

(инициалы и фамилия)

(краткое наименование раздела)

(подпись и дата подписания)

(инициалы и фамилия)

(краткое наименование раздела)

(подпись и дата подписания)

(инициалы и фамилия)

РЕФЕРАТ

Отчет 49 страниц, 14 рисунков, 2 таблицы, 7 источников.

МЕССЕНДЖЕР, РАЗРАБОТКА, КОММУНИКАЦИЯ, ФУНКЦИОНАЛ, ИНТЕРФЕЙС, БЕЗОПАСНОСТЬ, ТЕХНОЛОГИИ, ПОЛЬЗОВАТЕЛИ, ПРОИЗВОДИТЕЛЬНОСТЬ.

Целью дипломного проекта является разработать функциональный и удобный мессенджер, предназначенный для общения игроков в реальном времени.

ABSTRACT

Report 49 pages, 14 pictures, 2 tables, 7 references.

MESSENGER, DEVELOPMENT, COMMUNICATION,
FUNCTIONALITY, INTERFACE, SECURITY, TECHNOLOGY, USERS,
PERFORMANCE.

The purpose of the thesis is to develop a functional and user-friendly messenger designed for real-time communication between players.

СОДЕРЖАНИЕ

	Лист
ВВЕДЕНИЕ	4
1 ПОСТАНОВКА ЦЕЛЕЙ И ЗАДАЧ	5
1.1 Актуальность разработки	5
1.2 Цель работы	5
1.3 Задачи	5
2 ОБЗОР АНАЛОГОВ	6
2.1 Discord	6
2.2 TeamSpeak	6
2.3 Mumble	7
2.4 Overtone	7
2.5 Сравнительный анализ	8
3 ВЫБОР ИНСТРУМЕНТОВ РЕАЛИЗАЦИИ	9
3.1 Язык программирования	9
3.2 Инструмент для разработки клиентской части	10
3.3 Фреймворк для клиент-серверного взаимодействия	11
3.4 Система управления базами данных	13
4 ПРОЦЕСС РАЗРАБОТКИ	15
4.1 Создание клиент-серверного приложения	15
4.2 Реализация сервера и базы данных	18
4.3 Формы регистрации и авторизации	20
4.4 Главная форма	26
4.5 Взаимодействия между пользователями	29
4.6 Реализация комнат	33
5 ЭКОНОМИЧЕСКАЯ ЧАСТЬ	44
5.1 Экономические затраты на разработку проекта	44
5.2 Капитальные затраты на внедрение программы	47
5.3 Экономический эффект от применения программы	47
ЗАКЛЮЧЕНИЕ	48
СПИСОК ЛИТЕРАТУРЫ	49

					ДП 09.02.07. 10. Д. ПЗ			
Изм.	Лист	№ докум.	Подп.	Дата	Разработка мессенджера «GameChat» КТИ (филиал) ВолгГТУ			
Разраб.		Костюк И.В.		11.06.25				
Пров.		Дегтярёв А.Д.		11.06.25				
Н.контр.		Ромашенко А.И.		11.06.25				
Утв.		Харитонов И.М.		11.06.25				
					Лит.	Лист	Листов	
					У	3	49	

ВВЕДЕНИЕ

В последние годы отмечается значительное увеличение популярности онлайн-игр и киберспорта, связанное с развитием информационных технологий и ростом числа активных пользователей, вовлеченных в виртуальные миры. Важным элементом игрового процесса становится коммуникация между пользователями, позволяющая не только координировать действия внутри игры, но и формировать социальные связи, а также обмениваться опытом и впечатлениями. Однако современные средства коммуникации в играх зачастую обладают недостаточной функциональностью и интеграцией. Поэтому разработка специализированного игрового мессенджера «GameChat» представляет собой актуальную задачу. Данный мессенджер будет включать в себя функции, ориентированные на нужды игроков, такие как текстовый чат, создание групповых чатов и поддержку различных игровых жанров. Это позволит повысить качество взаимодействия между участниками и улучшить их игровой опыт.

					ДП 09.02.07. 10. Д. ПЗ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		4

1 ПОСТАНОВКА ЦЕЛЕЙ И ЗАДАЧ

1.1 Актуальность разработки

В условиях цифровых технологий и стремительного развития игровой индустрии коммуникация между игроками становится важным аспектом игрового процесса. Игровые мессенджеры, позволяющие пользователям обмениваться сообщениями, находить единомышленников и обсуждать стратегии, приобретают все большую популярность. Однако существующие решения зачастую не удовлетворяют потребности игроков в удобстве, функциональности и безопасности. Таким образом, разработка специализированного игрового мессенджера является актуальной и востребованной задачей.

1.2 Цель работы

Целью данной работы является обеспечение комфортной и эффективной коммуникации между игроками внутри игровых сообществ, командных игр и многопользовательских платформ. Мессенджер позволит игрокам обмениваться сообщениями, координировать свои действия в реальном времени, делиться стратегиями и достижениями, а также создавать и поддерживать дружеские связи вне игрового процесса.

1.3 Задачи

Для достижения поставленной цели необходимо решить ряд задач:

- проектирование архитектуры приложения;
- создание пользовательского интерфейса;
- разработка функционала: добавление пользователей в друзья, обмен сообщениями, создание групповых каналов по категориям игр;
- обеспечение безопасности: данных внедрить методы защиты данных пользователей (шифрование, аутентификация и т.д.).

Выполнение указанных задач позволит не только достичь поставленной цели, но и создать качественный и востребованный продукт, соответствующий современным требованиям пользователей в сфере онлайн-игр.

2 ОБЗОР АНАЛОГОВ

В данной главе представлен обзор существующих аналогов игровых мессенджеров, их сильные и слабые стороны, а также базовый функционал, который они предлагают. Это позволит сформировать представление о том, какие функции являются наиболее востребованными среди пользователей, а также выявить потенциальные проблемы, которые могут быть устранены при разработке игрового мессенджера «GameChat».

2.1 Discord

Discord — это платформа для общения, созданная в 2015 году, которая изначально ориентировалась на геймеров, но со временем расширила свою аудиторию. Она предлагает текстовые, голосовые и видеозвонки, а также возможность создания серверов для различных сообществ.

Ключевые функциональные особенности:

- создание текстовых и голосовых каналов;
- обмен сообщениями и файловыми вложениями;
- интеграция с различными игровыми платформами и сервисами;
- возможность использования пользовательских ботов для автоматизации задач и расширения функциональности;
- настройка серверов под специфические нужды пользователей.

Преимущества:

- высокое качество связи;
- низкая задержка;
- возможность создания многоуровневых сообществ;
- широкие возможности кастомизации серверов и каналов.

Недостатки:

- ограниченная функциональность на бесплатных тарифах;
- сложность в управлении большими сообществами, особенно при отсутствии модераторов.

2.2 TeamSpeak

TeamSpeak — это программа для голосового общения, разработанная в 2001 году, которая ориентирована для игроков в многопользовательские игры. Она позволяет пользователям создавать собственные сервера для общения в реальном времени.

Ключевые функциональные особенности:

- надежная система голосовых коммуникаций с поддержкой множества участников;
- возможность создания и управления собственными серверами;
- использование шифрования для защиты передаваемых данных;
- поддержка плагинов и скриптов для расширения функционала.

Преимущества:

- высокая степень безопасности благодаря использованию шифрования;
 - гибкость настройки серверов под конкретные нужды.
- Недостатки:
- сложность настройки и администрирования серверов для неподготовленных пользователей;
 - интерфейс может показаться устаревшим по сравнению с современными решениями.

2.3 Mumble

Mumble – это бесплатное и открытое программное обеспечение для голосового общения, созданное в 2005 году. Оно широко используется геймерами и другими пользователями, которым требуется высококачественное голосовое общение в реальном времени.

Ключевые функциональные особенности:

- возможность привязать звук к местоположению игрока в игре;
- шифрование всех передаваемых данных для обеспечения конфиденциальности;
- разграничение прав доступа для разных групп пользователей.

Преимущества:

- бесплатное использование без ограничений;
- открытый исходный код, позволяющий вносить изменения и улучшения;
- низкая задержка и высокое качество звука голосового чата.

Недостатки:

- требует установки и настройки сервера, что может быть сложным для новичков;
- ограниченный функционал по сравнению с коммерческими продуктами.

2.4 Overtone

Overtone – это относительно новый игровой мессенджер, созданный компанией Vivox Inc. в 2018 году. Он предлагает голосовую связь высокого качества и интеграцию с популярными играми.

Ключевые функциональные особенности:

- автоматическое подключение к игровому процессу и настройка параметров связи;
- хранение настроек и истории сообщений в облаке.

Преимущества:

- интуитивно понятный интерфейс и минимальные требования к настройке;
- быстрая интеграция с популярными играми;
- высококачественная голосовая связь с низким уровнем задержки.

Недостатки:

- ограниченное количество поддерживаемых игр по сравнению с конкурентами;

- недостаточно развитые возможности кастомизации.

2.5 Сравнительный анализ

Основная цель данного анализа — выделить ключевые функциональные особенности, преимущества и недостатки игрового мессенджера «GameChat», а также провести сравнение с другими популярными аналогами на рынке. Это позволит определить, какие решения наиболее успешны и как они могут быть применены для улучшения функциональности «GameChat».

Все рассмотренные аналоги предоставляют базовые функции, необходимые для общения в игровом процессе, такие как текстовые и голосовые каналы, возможность обмена сообщениями и интеграция с играми. Однако некоторые из них выделяются дополнительными функциями, которые могут значительно улучшить пользовательский опыт:

- Discord: включает возможность создания многоуровневых серверов и интеграцию с различными играми, а также поддержку ботов для автоматизации задач.
- TeamSpeak: предлагает высокое качество звука и низкую задержку, что делает его идеальным для командных игр, а также возможность записи разговоров.
- Overtone: имеет удобный интерфейс для управления проектами и интеграцию с другими инструментами, что позволяет эффективно организовывать рабочие процессы.

На основе проведенного анализа можно сделать следующие выводы:

- Удобный и понятный интерфейс. Удобный интерфейс, как в аналоге Overtone, способствует улучшению пользовательского опыта и может привлечь больше пользователей к «GameChat».
- Низкая задержка. Как показано в аналоге TeamSpeak, эта характеристика критически важна для командных игр, и ее реализация в «GameChat» повысит удовлетворенность пользователей.
- Настройка прав доступа и безопасность. Функции, представленные в аналоге Mumble, могут помочь в управлении большими игровыми сообществами и обеспечении безопасности пользователей.

Таким образом, при разработке и улучшении функциональности «GameChat» следует учитывать наиболее успешные решения, выявленные в анализе аналогов, чтобы создать удобный и многофункциональный мессенджер, способствующий развитию игрового сообщества и улучшению взаимодействия между игроками.

3 ВЫБОР ИНСТРУМЕНТОВ РЕАЛИЗАЦИИ

В данной главе мы рассмотрим выбор инструментов для реализации игрового мессенджера «GameChat». Мессенджер будет реализован в виде клиент-серверного приложения, которое включает в себя две программы: серверную программу, принимающую и обрабатывающую запросы от клиентского приложения, и клиентское приложение, в котором будет реализован весь функционал работы мессенджера. Для достижения поставленных целей необходимо тщательно подобрать инструменты, которые обеспечат высокое качество работы и удобство использования.

3.1 Язык программирования

3.1.1 C# — современный объектно-ориентированный язык программирования, разработанный компанией Microsoft. Язык C# практически универсален: его можно использовать для создания любого программного обеспечения: продвинутых бизнес-приложений, видеоигр, функциональных веб-приложений, приложений для Windows, macOS, мобильных программ для iOS и Android.

Достоинства:

- высокая производительность благодаря компиляции в промежуточный код IL (Intermediate Language);
- поддержка многопоточности и асинхронного программирования;
- интеграция с другими технологиями Microsoft, такими как WPF (Windows Presentation Foundation), ASP.NET (Active Server Pages для .NET) и WCF (Windows Communication Foundation).

Недостатки:

- ограниченная кроссплатформенность без использования дополнительных технологий вроде Mono.

3.1.2 Java — мультифункциональный объектно-ориентированный язык программирования со строгой типизацией. Разработан компанией Sun Microsystems (в последующем приобретённой компанией Oracle). Java используется для написания кода, который может выполняться на разных платформах: компьютерах, мобильных устройствах и серверах.

Достоинства:

- кроссплатформенность благодаря виртуальной машине Java (JVM);
- большое количество библиотек и фреймворков;
- возможность написания кода на стороне сервера и клиента.

Недостатки:

- более высокая сложность развертывания и настройки по сравнению с C#.

3.1.3 Python — высокоуровневый язык программирования общего назначения с динамической строгой типизацией и автоматическим управлением памятью. Ориентирован на повышение производительности разработчика,

читаемости кода и его качества, а также на обеспечение переносимости написанных на нём программ.

Достоинства:

- простота освоения и легкость чтения кода;
- огромная библиотека модулей и пакетов для различных задач;
- подходит для быстрого прототипирования.

Недостатки:

- низкая производительность по сравнению с компилируемыми языками.

После тщательного рассмотрения всех вариантов, был выбран язык C#. Этот выбор обусловлен несколькими ключевыми факторами:

- благодаря компиляции в промежуточный код IL, C# обеспечивает высокую производительность, что особенно важно для клиент-серверных приложений, где требуется быстрая обработка запросов и обмен данными;
- функции многопоточности и асинхронизации позволяют эффективно управлять ресурсами и обеспечивать быструю реакцию на запросы пользователей, что критично для мессенджеров;
- C# предоставляет широкий набор стандартных библиотек и инструментов, что упрощает разработку и ускоряет процесс создания приложений.

Таким образом, C# является оптимальным выбором для разработки клиент-серверного приложения, учитывая его производительность, функциональность и поддержку современными инструментами разработки.

3.2 Инструмент для разработки клиентской части

3.2.1 Windows Forms — это платформа пользовательского интерфейса для создания классических приложений Windows. Она поддерживает широкий набор функций для разработки приложений, включая элементы управления, графику, привязку данных и ввод пользователя.

Достоинства:

- простота в использовании;
- быстрая разработка интерфейсов;
- хорошая интеграция с .NET.

Недостатки:

- ограниченные возможности для создания современных интерфейсов;
- отсутствие поддержки для некоторых современных UI-паттернов (User Interface Patterns).

3.2.2 WPF (Windows Presentation Foundation) — система для построения клиентских приложений Windows с визуально привлекательными возможностями взаимодействия с пользователем. С помощью WPF можно создавать широкий спектр как автономных, так и запускаемых в браузере приложений.

Достоинства:

- более современные возможности для создания интерфейсов;
- поддержка XAML (Extensible Application Markup Language);

- возможность работы с векторной графикой.

Недостатки:

- более высокая сложность в освоении по сравнению с Windows Forms;
- требует больше ресурсов.

3.2.3 Avalonia — кроссплатформенная платформа пользовательского интерфейса на основе XAML. Avalonia позволяет писать на C# приложения под Windows, Linux и macOS. Начиная с Avalonia 11 preview 1 появилась возможность запуска на iOS, Android и Web (без Blazor) в экспериментальном состоянии.

Достоинства:

- кроссплатформенность;
- открытый исходный код;
- лёгкость и эффективность.

Недостатки:

- по сравнению с более устоявшимися фреймворками, такими как WPF или WinForms, в Avalonia может быть меньше функций и проблем со стабильностью;
- для разработчиков доступно меньше сторонних библиотек, инструментов и компонентов.

Выбор Windows Forms для разработки клиентской части был оправдан по нескольким причинам:

- windows Forms предлагает более простой и интуитивно понятный интерфейс для разработки, что позволяет быстрее создавать пользовательские интерфейсы;
- для разработчиков, которые уже знакомы с C#, Windows Forms может быть более доступным вариантом. Это позволяет избежать дополнительных затрат времени на изучение более сложных технологий, таких как WPF или Avalonia;
- windows Forms является зрелой технологией, которая существует уже много лет. Она хорошо документирована и поддерживается, что упрощает поиск решений для возникающих проблем;
- windows Forms приложения имеют меньшие системные требования по сравнению с WPF и Avalonia, что может быть важным для пользователей с устаревшими или менее мощными устройствами;
- windows Forms приложения проще развертывать и распространять, так как они могут быть упакованы в один исполняемый файл, что упрощает процесс установки для конечных пользователей.

Windows Forms на C# для создания клиентской части мессенджера выбран благодаря своей простоте, скорости разработки и стабильности, что делает его подходящим инструментом для реализации данного проекта.

3.3 Фреймворк для клиент-серверного взаимодействия

3.3.1 SignalR — библиотека для реализации двунаправленной связи в реальном времени между клиентом и сервером на основе WebSocket (независимый протокол связи поверх TCP-соединения).

Достоинства:

- обеспечивает высокую интерактивность и поддержку push-уведомлений;
- работает поверх существующих HTTP-соединений;
- интегрируется с ASP.NET (Active Server Pages для .NET).

Недостатки:

- больше ориентирован на веб-приложения, чем на классические клиентские программы;
- требует дополнительного обучения и настройки.

3.3.2 WCF (Windows Communication Foundation) — это программный фреймворк, используемый для обмена данными между приложениями, входящий в состав .NET Framework. Он делает возможным построение безопасных и надёжных транзакционных систем через упрощённую унифицированную программную модель межплатформенного взаимодействия.

Достоинства:

- гибкость в выборе транспортных протоколов (HTTP, TCP, Named Pipes и др.);
- встроенная поддержка безопасности и аутентификации;
- легкое создание и использование сервисов.

Недостатки:

- может показаться сложным для новичков из-за большого количества настроек и конфигурационных файлов;
- ограничена платформой Windows.

3.3.3 gRPC (Remote Procedure Calls) — это система удалённого вызова процедур (RPC) с открытым исходным кодом, первоначально разработанная в Google в 2015 году. В нём клиентское приложение может напрямую вызывать метод серверного приложения на другом компьютере, как если бы это был локальный объект. Это упрощает создание распределённых приложений и служб.

Достоинства:

- высокая скорость и эффективность за счет использования бинарного протокола Protobuf;
- поддерживает потоковую передачу данных;
- кроссплатформенность.

Недостатки:

- более сложная настройка по сравнению с WCF;
- отсутствие встроенной поддержки в .NET Framework (требуется переход на .NET Core).

Учитывая требования проекта, был выбран фреймворк WCF. Он идеально подходит для разработки клиент-серверных приложений на платформе Windows, обеспечивая высокую гибкость и безопасность. Кроме того, WCF хорошо интегрируется с языком C# и позволяет легко создавать сервисы, соответствующие требованиям мессенджера.

3.4 Система управления базами данных

3.4.1 MySQL — это свободная реляционная система управления базами данных (СУБД). Она имеет открытый исходный код, поддерживая множество операционных систем. Разработку и поддержку MySQL осуществляет корпорация Oracle.

Достоинства:

- бесплатная лицензия и открытый исходный код;
- широкое сообщество и большое количество документации;
- кроссплатформенность.

Недостатки:

- относительно меньшие возможности по сравнению с коммерческими решениями;
- могут возникнуть проблемы с производительностью при работе с большими объемами данных.

3.4.2 SQL Server Management Studio — это бесплатная графическая среда, включающая набор инструментов для разработки сценариев на T-SQL (Transact-SQL — это расширение языка SQL) и управления инфраструктурой Microsoft SQL Server.

Достоинства:

- глубокая интеграция с экосистемой Microsoft;
- мощные средства аналитики и отчетности;
- надежность и масштабируемость.

Недостатки:

- ограниченность платформы Windows;
- возможны сложности с лицензированием в коммерческих проектах.

3.4.3 MongoDB — это документоориентированная система управления базами данных с открытым исходным кодом. Для хранения данных используется JSON-подобный формат.

Достоинства:

- гибкость схемы данных;
- высокая производительность при работе с большими объемами данных;
- простота масштабирования.

Недостатки:

- отсутствие строгих транзакций и поддержки ACID (atomicity, consistency, isolation, durability — набор требований к транзакционной системе);
- необходимость изменения подхода к проектированию базы данных по сравнению с реляционными СУБД.

Исходя из требований проекта, была выбрана система управления базами данных SQL Server Management Studio. Она предлагает наилучшую интеграцию с платформой Windows и языком C#, а также обеспечивает высокую надежность и производительность, что критически важно для функционирования игрового мессенджера «GameChat».

В результате проведенного анализа было принято решение использовать следующие инструменты для реализации игрового мессенджера «GameChat»:

- язык программирования: C#;

- инструмент для разработки клиентской части: Windows Forms;
- фреймворк для клиент-серверного взаимодействия: WCF;
- система управления базами данных: SQL Server Management Studio.

Эти инструменты обеспечивают наилучшее сочетание производительности, надежности и удобства разработки, что позволит создать качественный и эффективный продукт.

4 ПРОЦЕСС РАЗРАБОТКИ

4.1 Создание клиент-серверного приложения

Для реализации игрового мессенджера «GameChat» используется клиент-серверная архитектура приложения. Клиент-серверная архитектура — это распределённая архитектура, в которой задачи выполняются на двух типах устройств: клиенте и сервере. Клиент — это устройство или приложение, обращающееся к серверу за выполнением определённой задачи или получением ресурсов. Сервер — устройство или программа, предоставляющая запрашиваемые ресурсы и обработку данных. На рисунке 1 представлена схема клиент-серверной архитектуры.

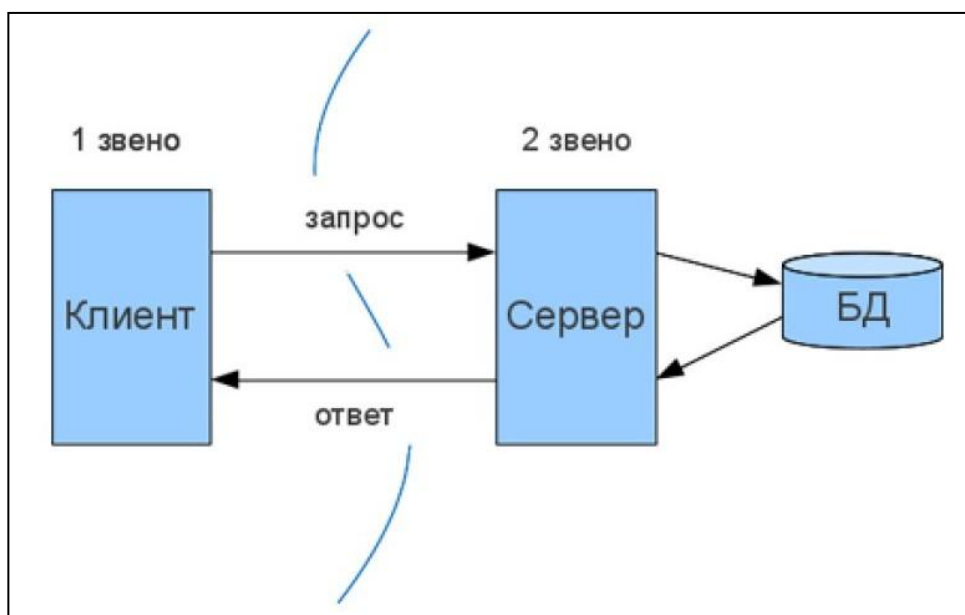


Рисунок 1 — Схема клиент-серверной архитектуры

Таким образом, реализовано две программы: серверная и клиентская, и связаны между собой по протоколу WCF (Windows Communication Foundation). WCF — это платформа для создания приложений, ориентированных на обслуживание. Это среда выполнения и набор API для создания систем, которые отправляют сообщения между службами и клиентами. Она позволяет создавать безопасные, надёжные и транзакционные веб-сервисы, которые интегрируются между платформами и технологиями. С помощью WCF можно предоставлять услуги по различным протоколам, таким как HTTP, TCP, MSMQ и другим. Исходя из этого, данный мессенджер использует протокол HTTP для обмена метаданными, а сами сервисы работают по протоколу TCP, так как HTTP обеспечивает удобный и универсальный доступ к метаданным, а TCP предлагает надёжную и высокопроизводительную среду для обмена данными между сервисами.

Ниже приведён код программы сервера, при помощи которого сервер связывается с клиентским приложением:

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
```

```

<startup>
<supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.8" />
</startup>
<system.serviceModel>
<bindings>
<netTcpBinding>
<binding name="LargeMessageBinding"
maxReceivedMessageSize="2147483647">
<security mode="None">
<transport clientCredentialType="None" />
</security>
</binding>
</netTcpBinding>
</bindings>
<behaviors>
<serviceBehaviors>
<behavior name="mexBeh">
<serviceMetadata httpGetEnabled="true" httpsGetEnabled="true" />
<serviceDebug includeExceptionDetailInFaults="true" />
</behavior>
</serviceBehaviors>
</behaviors>
<services>
<service behaviorConfiguration="mexBeh" name="WCF.Service">
<endpoint address="" binding="netTcpBinding"
bindingConfiguration="LargeMessageBinding"
contract="WCF.IService" />
<endpoint address="mex" binding="mexHttpBinding"
contract="IMetadataExchange" />
<host>
<baseAddresses>
<add baseAddress="http://ip:port" />
<add baseAddress="net.tcp://ip:port" />
</baseAddresses>
</host>
</service>
<service behaviorConfiguration="mexBeh" name="WCF.Service2">
<endpoint address="" binding="netTcpBinding"
bindingConfiguration="LargeMessageBinding"
contract="WCF.IService2" />
<endpoint address="mex" binding="mexHttpBinding"
contract="IMetadataExchange" />
<host>
<baseAddresses>
<add baseAddress="http://ip:port" />
<add baseAddress="net.tcp://ip:port" />

```

```

</baseAddresses>
</host>
</service>
</services>
</system.serviceModel>
</configuration>

```

В программе используется два сервиса. Первый — «Service», в котором происходят основные функции, такие как авторизация и регистрация пользователей, добавление и удаление друзей, создание и удаление комнат и так далее. Второй сервис — «Service2», который используется для принятия данных от одного пользователя и отправкой данных другим конкретным клиентам. Например: отображение статуса пользователя (в сети, не в сети); отправка сообщений в комнатах и между друзьями; блокировка пользователей в комнатах.

Ниже представлен код, по которому клиентское приложение связывается с серверной программой:

```

<?xml version="1.0" encoding="utf-8" ?>
<configuration>
<startup>
<supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.8" />
</startup>
<system.serviceModel>
<bindings>
<netTcpBinding>
<binding
name="NetTcpBinding_IService"
maxReceivedMessageSize="2147483647">
<security mode="None">
<transport sslProtocols="None" />
</security>
</binding>
<binding name="NetTcpBinding_IService2">
<security mode="None">
<transport sslProtocols="None" />
</security>
</binding>
</netTcpBinding>
</bindings>
<client>
<endpoint address="net.tcp://ip:port/" binding="netTcpBinding"
bindingConfiguration="NetTcpBinding_IService2"
contract="Service2.IService2"
name="NetTcpBinding_IService2" />
<endpoint address="net.tcp://ip:port/" binding="netTcpBinding"
bindingConfiguration="NetTcpBinding_IService" contract="Server.IService"
name="NetTcpBinding_IService" />
</client>

```



```
</system.serviceModel>  
</configuration>
```

4.2 Реализация сервера и базы данных

В качестве сервера используется SQL Server — это система управления реляционными базами данных (СУБД), разработанная компанией Microsoft. Она используется для хранения, управления и поиска данных в структурированном и организованном виде. Для управления SQL Server используется инструмент SQL Server Management Studio (SSMS) — это программное приложение, используемое для управления, настройки и администрирования Microsoft SQL Server. Он предоставляет графический интерфейс для работы с SQL Server, позволяя пользователям создавать базы данных и управлять ими, запускать запросы, а также разрабатывать и отлаживать хранимые процедуры.

В SQL Server Management Studio была создана база данных, которая должна хранить в себе:

- информацию об учётных записях пользователей: имя пользователя, адрес электронной почты, пароль, статус (в сети, не в сети);
- информацию о добавлении пользователей в друзья: пользователь который отправил запрос в друзья, пользователь кому отправили запрос, статус запроса (отправлен, принят, отклонен);
- информацию о переписках пользователей: пользователь который отправил сообщение, пользователь которому пришло сообщение, содержимое сообщения, дата и время отправки сообщения, статус сообщения (прочитано, не прочитано);
- информацию об играх по которым сортируются комнаты: название и изображение игры;
- информацию о комнатах: название комнаты, игра в которой создана комната, пароль и количество участников;
- информацию об участниках комнат: комната где участвует пользователь, сам пользователь, статус (создатель, участник, выгнан);
- информацию о переписках в комнатах: комната в которой отправили сообщение, пользователь который отправил сообщение, сообщение, дата и время отправки сообщения.

Исходя из этого были выявлены следующие сущности и их атрибуты:

1) Users:

- user_id: int (Primary key);
- user_name: varchar(50);
- email: varchar(100);
- password: varchar(50);
- status: varchar(10).

2) Friends:

- friends_id: int (Primary key);
- userSender_id: int (Foreign key);
- userRecipient_id: int (Foreign key);
- status: varchar(15).

- 3) FriendMessages:
 - message_id: int (Primary key);
 - sender_id: int (Foreign key);
 - recipient_id: int (Foreign key);
 - message: text;
 - message_date: datetime;
 - status: varchar(15).
- 4) Games:
 - game_id: int (Primary key);
 - game_name: varchar(50);
 - game_image: image.
- 5) Rooms:
 - room_id: int (Primary key);
 - game_id: int (Foreign key);
 - room_name: varchar(50);
 - password: varchar(50);
 - members_current: int;
 - members_max: int.
- 6) RoomMembers:
 - roomMember_id: int (Primary key);
 - user_id: int (Foreign key);
 - room_id: int (Foreign key);
 - status: varchar(20).
- 7) RoomMessages:
 - roomMessage_id: int (Primary key);
 - room_id: int (Foreign key);
 - sender_id: int (Foreign key);
 - message: text;
 - message_date: datetime.

На рисунке 2 изображена ER-модель базы данных игрового мессенджера «GameChat».

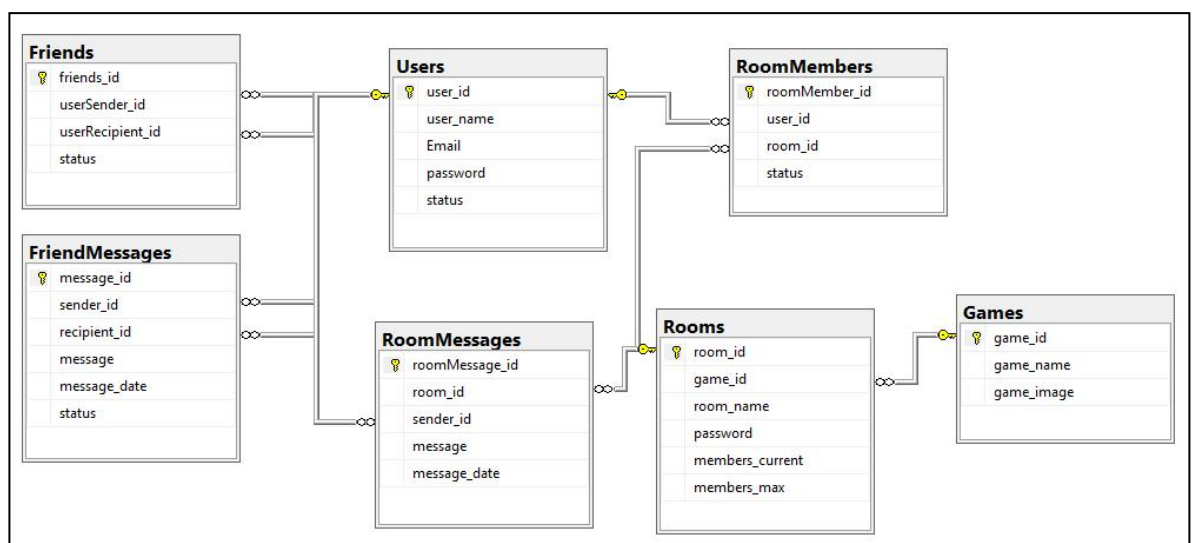


Рисунок 2 — ER-модель базы данных «GameChat»

4.3 Формы регистрации и авторизации

При первом запуске программы нужно зарегистрировать учётную запись. На рисунке 3 представлен пользовательский интерфейс для регистрации учётных записей.

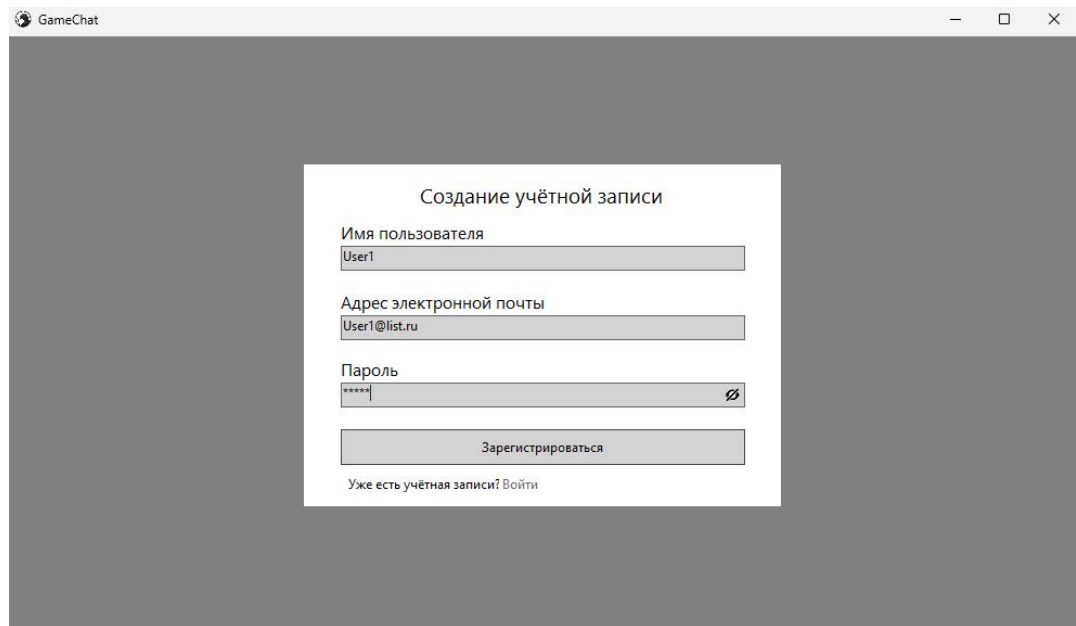


Рисунок 3 — Форма «Создание учётной записи»

При помощи данной формы пользователи могут создать учётную запись, введя адрес электронной почты, имя пользователя и пароль. Под каждым полем есть строка для ошибки, которая появляется, когда имя пользователя или пароль по количеству символов меньше допустимого, или если адрес электронной почты не существует:

```
if (textBoxUserName.Text.Length < 4)
{
    labelErrorUserName.Text = "Имя не может содержать менее 4  
символов";
    labelErrorUserName.Visible = true;
}
else if (textBoxPassword.Text.Length < 6)
{
    labelErrorPassword.Text = "Пароль не может быть меньше 6 символов";
    labelErrorPassword.Visible = true;
}
else
{
    client = new ServiceClient();
    string reg = client.accountCheckForReg(textBoxUserName.Text,  
textBoxEmail.Text);
    if (reg == "аккаунта не существует")
    {
        SendCode();
    }
}
```

```

        if (errorEmail == "")
        {
            Account account = new Account(textBoxUserName.Text, textBoxEmail.Text,
textBoxPassword.Text);
            screen.replacmentAccountConfirmationForm(code, account);
        }
        else
        {
            labelErrorEmail.Text = "неверный Email";
            labelErrorEmail.Visible = true;
        }
    }
    else if (reg == "Имя занято")
    {
        labelErrorUserName.Text = reg;
        labelErrorUserName.Visible = true;
    }
    else if (reg == "Email занят")
    {
        labelErrorEmail.Text = reg;
        labelErrorEmail.Visible = true;
    }
}

```

Для проверки существующего адреса электронной почты используется метод, который пытается отправить на почту код для подтверждения:

```

private void SendCode()
{
    errorEmail = "";
    code = "";
    Random number = new Random();
    for (int i = 0; i < 8; i++) code += number.Next(0, 10).ToString();
    try
    {
        string from = "chat_game@mail.ru";
        string pass = password;
        SmtplibClient client = new SmtplibClient("smtp.mail.ru", 587);
        client.Credentials = new NetworkCredential(from, pass);
        client.EnableSsl = true;
        MailMessage mess = new MailMessage();
        mess.From = new MailAddress(from);
        mess.To.Add(textBoxEmail.Text.ToString());
        mess.Subject = "Подтверждение регистрации в ChatGame";
        mess.Body = $"Код для подтверждения учётной записи: {code} ";
        client.Send(mess);
    }
    catch (Exception e)
    {
    }
}

```

```

{
    errorEmail = e.Message;
}
}

```

Если код успешно отправлен значит адрес электронной почты существует, иначе отслеживается ошибка, которая обозначает что адрес электронной почты не существует и выводит сообщение пользователю.

После того как ввели правильно все данные и нажали на кнопку регистрации, клиентское приложение отправляет запрос серверной программе. Запрос проверяет существует ли уже учетная запись с такими данными:

```

public string accountCheckForReg(string userName, string Email)
{
    string answer;
    SqlConnection connection = new SqlConnection(@"Data
Source=DENNY\SERVER;Initial Catalog=GameChat;Integrated Security=True");
    connection.Open();
    SqlDataAdapter adapter = new SqlDataAdapter();
    DataTable table = new DataTable();
    string query = $"select user_name from Users where user_name =
    '{userName}'";
    SqlCommand cmd = new SqlCommand(query, connection);
    adapter.SelectCommand = cmd;
    adapter.Fill(table);
    if (table.Rows.Count == 0)
    {
        table = new DataTable();
        query = $"select Email from Users where Email = '{Email}'";
        cmd = new SqlCommand(query, connection);
        adapter.SelectCommand = cmd;
        adapter.Fill(table);
        if (table.Rows.Count == 0) answer = "аккаунта не существует";
        else answer = "Email занят";
    }
    else answer = "Имя занято";
    connection.Close();
    return answer;
}

```

Если имя или адрес электронной почты уже существует, то сервер возвращает приложению сообщение, что имя или адрес электронной почты занят. Иначе программа отправляет сообщение, что учетной записи с такими данными не существует и затем открывается форма для ввода кода подтверждения.

На рисунке 4 представлено как выглядит форма для подтверждения учётной записи.

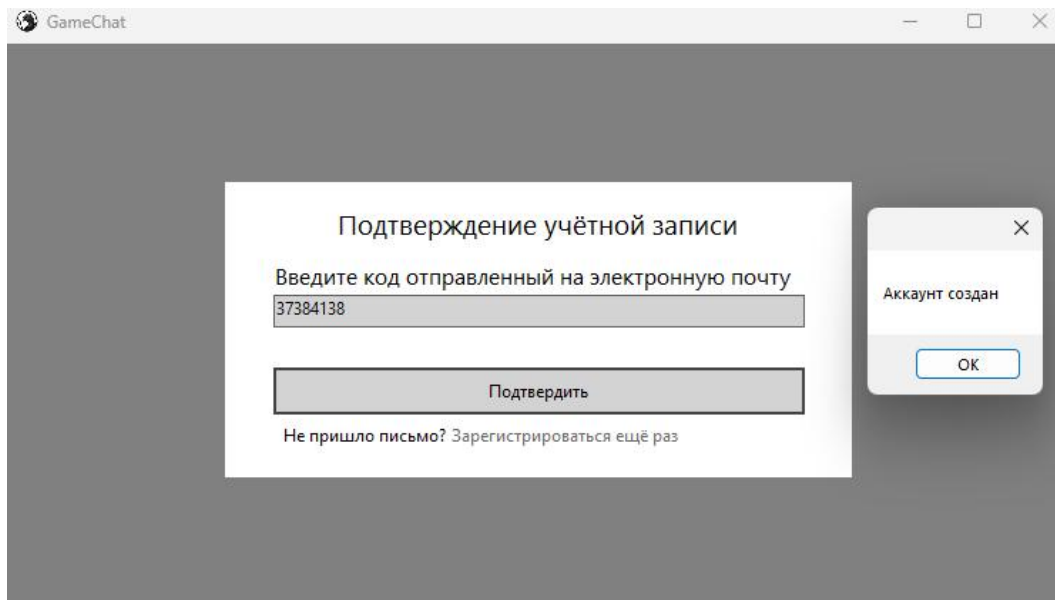


Рисунок 4 — Форма «Подтверждение учётной записи»

В данной форме нужно ввести код подтверждения, отправленный на электронную почту. Если код не был отправлен на указанную электронную почту, можно попробовать заново создать учётную запись, нажав на текст «Зарегистрироваться ещё раз».

Если код введён неверно, появится строка с ошибкой. Иначе если код введён верно, то после нажатия на кнопку приложение отправляет серверу данные для добавления их в базу данных:

```

if (textBoxCode.Text == code)
{
    client = new ServiceClient();
    string reg = client.Reg(account.userName, account.Email, account.password);
    if (reg == "аккаунт зарегистрирован")
    {
        MessageBox.Show("Аккаунт создан");
        screen.replacementLoginForm();
    }
    else
    {
        MessageBox.Show("ошибка");
        screen.replacementRegistrationForm();
    }
}
else
{
    labelErrorCode.Text = "Неверный код подтверждения";
    labelErrorCode.Visible = true;
}

```

Код для создания учётной записи в базе данных на серверной программе представлен ниже:

```

public string Reg(string userName, string Email, string password)

```

```

{
    string answer;
    SqlConnection connection = new SqlConnection(@"Data
Source=DENNY\SERVER;Initial Catalog=GameChat;Integrated Security=True");
    connection.Open();
    string query = $"insert into Users(user_name, Email, password, status)
values('{userName}', '{Email}', '{password}', 'не всему')";
    SqlCommand cmd = new SqlCommand(query, connection);
    if (cmd.ExecuteNonQuery() == 1) answer = "аккаунт зарегистрирован";
    else answer = "ошибка";
    connection.Close();
    return answer;
}

```

Если данные успешно добавлены в базу данных то появится сообщение об успешном создании учётной записи и откроется интерфейс для авторизации. На рисунке 5 представлена форма для авторизации учётной записи.

Рисунок 5 — Форма «Авторизация»

Данная форма отвечает за вход в учётную запись. Также, как и в интерфейсе для регистрации, имеется значок глаза, который отображает и скрывает пароль. Если нет созданной учётной записи, можно нажать на текст «Зарегистрироваться» внизу формы.

При нажатии на кнопку «Вход» отправляются данные на сервер для попытки входа в учётную запись:

```

private void buttonLogin_Click(object sender, EventArgs e)
{
    client = new ServiceClient();
    string log = client.login(textBoxEmail.Text, textBoxPassword.Text);
    if (log == "вход")
    {

```

```

screen.replacmentMainScreen(textBoxEmail.Text);
}
else if (log == "неверный Email")
{
labelErrorEmail.Text = log;
labelErrorEmail.Visible = true;
}
else if (log == "неправильный пароль")
{
labelErrorPassword.Text = log;
labelErrorPassword.Visible = true;
}
}
}

```

Код для попытки входа в учетную запись на сервере представлен ниже:

```

public string login(string Email, string password)
{
string answer;
SqlConnection connection = new SqlConnection(@"Data
Source=DENNY\SERVER;Initial Catalog=GameChat;Integrated Security=True");
connection.Open();
SqlDataAdapter adapter = new SqlDataAdapter();
DataTable table = new DataTable();
string query = $"select Email from Users where Email = '{Email}'";
SqlCommand cmd = new SqlCommand(query, connection);
adapter.SelectCommand = cmd;
adapter.Fill(table);
if (table.Rows.Count > 0)
{
table = new DataTable();
query = $"select Email, password from Users where Email = '{Email}' and
password = '{password}'";
cmd = new SqlCommand(query, connection);
adapter.SelectCommand = cmd;
adapter.Fill(table);
if (table.Rows.Count > 0) answer = "вход";
else answer = "неправильный пароль";
}
else answer = "неверный Email";
connection.Close();
return answer;
}

```

Если данные не совпадают с данными в базе данных, то выводится сообщение пользователю, что адрес электронной почты или пароль введены неверно. Иначе открывается главная форма мессенджера.

4.4 Главная форма

Перед отображением главной формы отправляется запрос на сервер для получения информации об учетной записи по адресу электронной почты. Если получить данные не удалось, то открывается форма авторизации и выводится сообщение об ошибке:

```
public void replacmentMainScreen(string Email)
{
    client = new ServiceClient();
    string Account = client.getAccount(Email);
    if (Account != "ошибка")
    {
        account = new Account(Account.Split()[0], Account.Split()[1],
Account.Split()[2]);
        client2 = new Service2Client(new InstanceContext(this));
        client2.Connect(account.userName);
        if (Size.Width < 990 + 16) Size = new Size(990 + 16, Size.Height);
        if (Size.Height < 550 + 39) Size = new Size(Size.Width, 550 + 39);
        MinimumSize = new Size(990 + 16, 550 + 39);
        mainScreen = new MainScreen(this, account, client2);
        mainScreen.Size = new Size(Width - 16, Height - 39);
        mainPanel.Controls.Clear();
        mainPanel.Controls.Add(mainScreen);
        activeForm = "MainScreen";
    }
    else
    {
        replacementLoginForm();
        MessageBox.Show(Account);
    }
}
```

Код для получения информации об учетной записи на сервере представлен ниже:

```
public string getAccount(string Email)
{
    string answer;
    SqlConnection connection = new SqlConnection(@"Data
Source=DENNY\SERVER;Initial Catalog=GameChat;Integrated Security=True");
    connection.Open();
    SqlDataAdapter adapter = new SqlDataAdapter();
    DataTable table = new DataTable();
    string query = $"select user_name, Email, password from Users where Email
= '{Email}'";
    SqlCommand cmd = new SqlCommand(query, connection);
    adapter.SelectCommand = cmd;
    adapter.Fill(table);
    if (table.Rows.Count > 0) answer = $"{table.Rows[0][0].ToString()}
```

```

{table.Rows[0][1].ToString()} {table.Rows[0][2].ToString()}";
    else answer = "ошибка";
    connection.Close();
    return answer;
}

```

При открытии главной формы используется второй сервис, который отмечает всех пользователей, открывших мессенджер, а также меняет статус пользователя на «в сети»:

```

public void Connect(string user_name)
{
    ServerUser user = new ServerUser()
    {
        user_name = user_name,
        operationContext = OperationContext.Current
    };
    users.Add(user);
    SqlConnection connection = new SqlConnection(@"Data
Source=DENNY\SERVER;Initial Catalog=GameChat;Integrated Security=True");
    connection.Open();
    string query = $"update Users set status = 'в сети' where user_name =
'{user_name}'";
    SqlCommand cmd = new SqlCommand(query, connection);
    cmd.ExecuteNonQuery();
    connection.Close();
}

```

Запись всех пользователей, которые открывают мессенджер, необходима для проведения действий на форме конкретных пользователей в реальном времени. Например, для появления сообщений у получателя, если в момент получения сообщения у него открыт чат, или для блокировки пользователей, тем самым закрывая и блокируя форму комнаты, если она открыта в момент блокировки.

После успешного получения информации об учетной записи открывается главная форма мессенджера. На рисунке 6 представлен интерфейс главной формы.

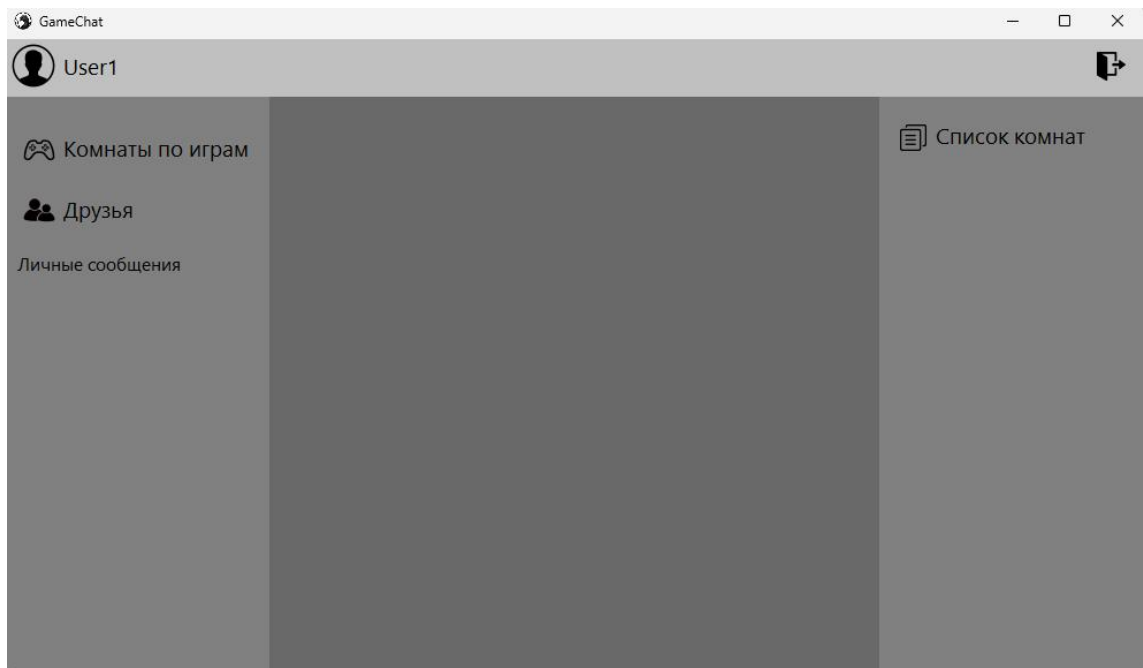


Рисунок 6 — Интерфейс главной формы

На главной форме находится 4 панели. Верхняя панель отображает имя пользователя и кнопку для выхода из аккаунта:

```
private void pictureBoxExit_Click(object sender, EventArgs e)
{
    screen.replacementLoginForm();
    clientSendMessage.Disconnect(account.userName);
}
```

При выходе из учетной записи, во втором сервисе пользователь вычеркивается из списка вошедших в мессенджер и статус меняется на «не в сети»:

```
public void Disconnect(string user_name)
{
    for (int i = 0; i < users.Count; i++)
    {
        if (users[i].user_name == user_name)
        {
            users.RemoveAt(i);
            SqlConnection connection = new SqlConnection(@"Data
Source=DENNY\SERVER;Initial Catalog=GameChat;Integrated Security=True");
            connection.Open();
            string query = $"update Users set status = 'не в сету' where user_name =
'{user_name}'";
            SqlCommand cmd = new SqlCommand(query, connection);
            cmd.ExecuteNonQuery();
            connection.Close();
        }
    }
}
```

На левой панели находятся две вкладки: вкладка «Комнаты по играм», где отображены игры в которых можно создавать комнаты, и вкладка «Друзья», которая отображает список добавленных пользователей в друзья, а также возможность принимать и отправлять запросы на дружбу другим пользователям. Снизу отображаются личные переписки с пользователями. Правая панель отображает список комнат в которых участвует пользователь. В центральной панели отображаются открытые вкладки, чаты и комнаты.

4.5 Взаимодействия между пользователями

На рисунке 7 представлен интерфейс вкладки «Друзья».

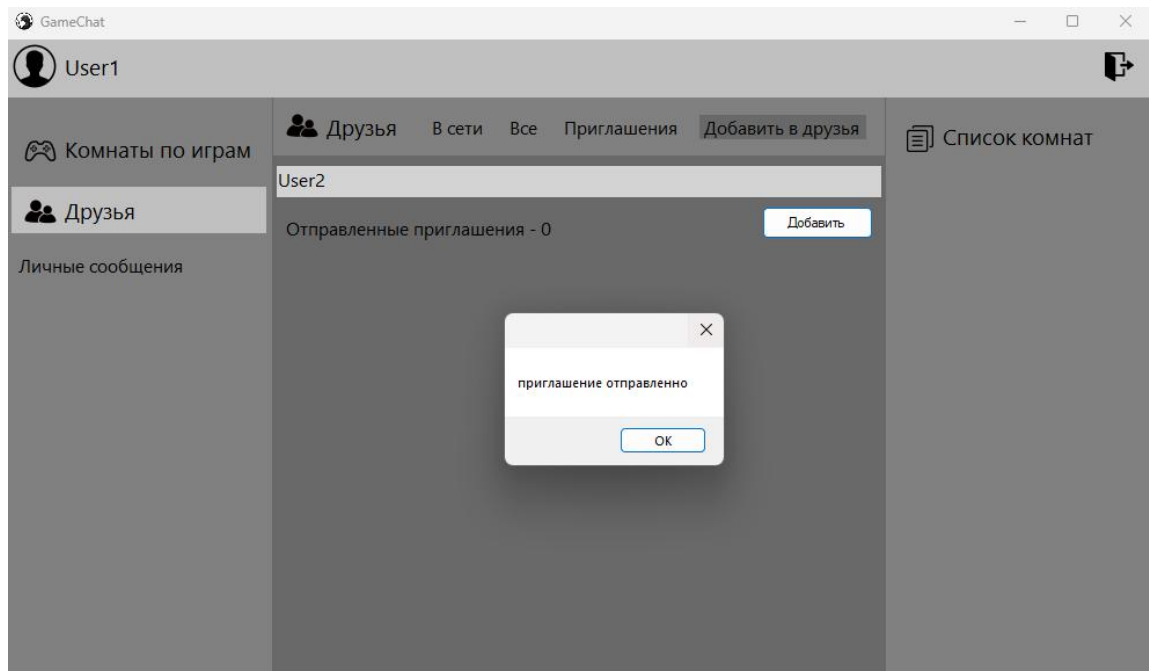


Рисунок 7 — Интерфейс вкладки «Друзья»

Данная вкладка отображает список друзей по категориям: «В сети» — отображает список друзей, которые сейчас находятся в мессенджере; «Все» — отображает список всех друзей; «Приглашения» — отображает список приглашений в друзья от других пользователей, которые можно либо принять, либо отклонить; «Добавить в друзья» — в этой вкладке можно отправлять запросы на дружбу другим пользователям, а также здесь отображаются все отправленные запросы в друзья, которые ещё не приняты.

В каждой вкладке есть поле «Поиск», при помощи которой можно найти нужного пользователя по его имени:

```
private void textBoxSearch_TextChanged(object sender, EventArgs e)
{
    if (activePanel != "FriendAdd" && textBoxSearch.Text != "Поиск")
    {
        client = new ServiceClient();
        List<string> friends = client.getFriendList(account.userName,
type).ToList<string>();
        for (int i = 0; i < friends.Count; i++)
        {
```

```

        if (!friends[i].StartsWith(textBoxSearch.Text))
        {
            friends.RemoveAt(i);
        }
    }
    FriendsListDisplay(friends);
}
if (activePanel != "FriendAdd" && textBoxSearch.Text == "" ||
textBoxSearch.Text == "Пуск")
{
    client = new ServiceClient();
    List<string> friends = client.getFriendList(account.userName,
type).ToList<string>();
    FriendsListDisplay(friends);
}
}
}

```

Когда открыта вкладка «Добавить в друзья», поиск заменяется на поле для ввода имени пользователя, которому нужно отправить запрос на добавления в друзья, а также появляется кнопка «Добавить» для отправки запроса. Если имя пользователя было введено неправильно или пользователь уже находится в друзьях, появиться предупреждение. Если имя пользователя введено правильно, то появиться сообщение, что запрос на добавление в друзья был отправлен:

```

private void buttonAddFriend_Click(object sender, EventArgs e)
{
    if (textBoxSearch.Text.Length < 4) MessageBox.Show("Имя не может
содержать менее 4 символов");
    else if (textBoxSearch.Text == account.userName) MessageBox.Show("это
ваше имя");
    else
    {
        client = new ServiceClient();
        string answer = client.FriendAdd(account.userName, textBoxSearch.Text);
        if (answer == "приглашение отправлено")
        {
            textBoxSearch.Text = "Введите имя пользователя";
            MessageBox.Show(answer);
            replacmentFriendAdd();
        }
        else MessageBox.Show(answer);
    }
}
}

```

Также при нажатии на пользователя в списке друзей откроется чат с этим пользователем:

```

public void replacmentChatFriend(string friend_name, messagePanel
messagePanel, List<string[]> messages)

```

```

{
    formReplacment();
    if (this.messagePanel == null) this.messagePanel = messagePanel;
    else
    {
        this.messagePanel.chatFriendClose();
        this.messagePanel = messagePanel;
    }
    chat = new Chat(this, account, client2, friend_name, messages);
    chat.Size = new Size(panelMain.Width, panelMain.Height);
    panelMain.Controls.Clear();
    panelMain.Controls.Add(chat);
    activePanelMain = "ChatFriend";
}

```

На рисунке 8 представлен интерфейс чата для личных сообщений с другим пользователем.

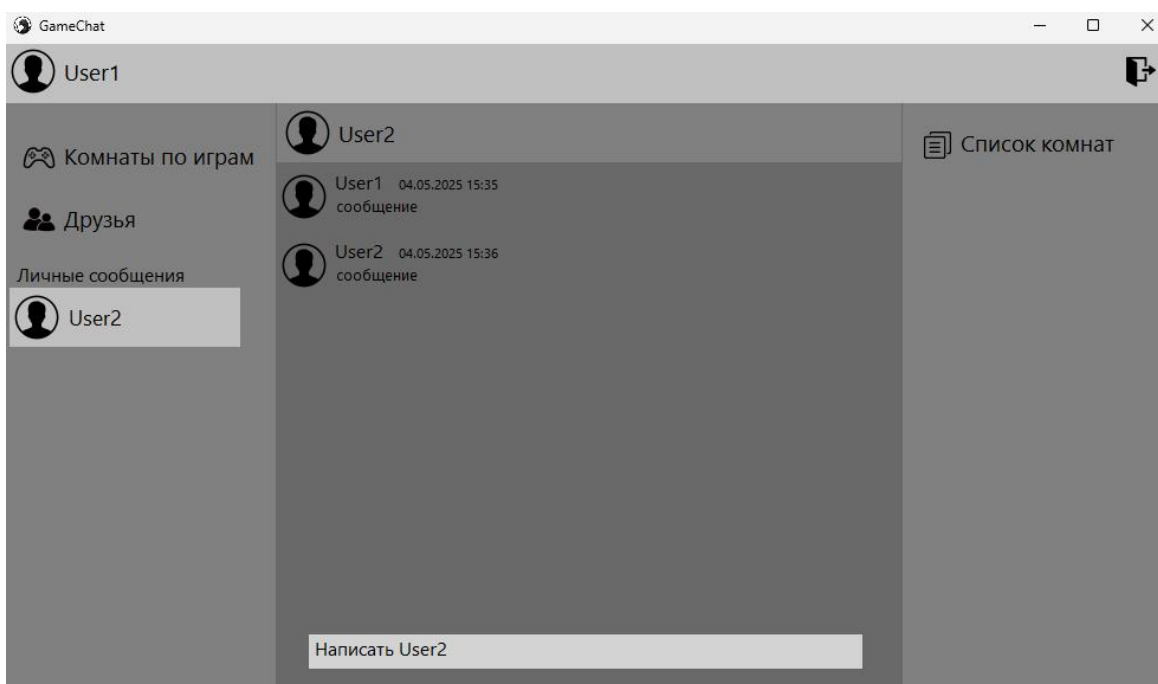


Рисунок 8 — Интерфейс чата

Сверху отображается имя пользователя, с которым открыт чат. Посередине находится панель, в которой отображаются сообщения между пользователями. Снизу находится поле для отправки сообщений:

```

private void textBoxMessage_KeyDown(object sender, KeyEventArgs e)
{
    bool probels = spaceStringCheck();
    if (e.KeyCode == Keys.Enter && probels == false)
    {
        string answer;
        string message = textBoxMessage.Text;
        DateTime dateTime = DateTime.Now;
    }
}

```

```

        answer = client2.SendMessageFriend(message, account.userName,
friendName, dateTime);
        if (answer == "сообщение отправлено")
        {
            sendMessage = true;
            SendMessagePanel sendMessagePanel = new
SendMessagePanel(account.userName, message, panelMessages.Controls.Count,
dateTime, this);
            if (panelMessages.Controls.Count == 0) sendMessagePanel.Size = new
Size(panelMessages.Width, sendMessagePanel.Height);
            else sendMessagePanel.Size = new
Size(panelMessages.Controls[panelMessages.Controls.Count - 1].Width,
sendMessagePanel.Height);
            if (panelMessages.Controls.Count == 0) sendMessagePanel.Location = new
Point(0, 0);
            else sendMessagePanel.Location = new Point(0,
panelMessages.Controls[panelMessages.Controls.Count - 1].Location.Y +
panelMessages.Controls[panelMessages.Controls.Count - 1].Height);
            panelMessages.Controls.Add(sendMessagePanel);
            mainScreen.repositionMessagePanel(friendName);
            sendMessage = false;
        }
        else if (answer == "ошибка") MessageBox.Show("ошибка, сообщение не
отправлено");
        quitTextBoxMessage(e, false);
    }
    else if (e.KeyCode == Keys.Enter && probels)
    {
        quitTextBoxMessage(e);
    }
    if (e.Control && e.KeyCode == Keys.V)
    {
        e.SuppressKeyPress = true;
        string clipboardText = Clipboard.GetText();
        clipboardText = clipboardText.Replace("\r\n", "").Replace("\n", "");
        textBoxMessage.AppendText(clipboardText);
    }
}
}

```

При нажатии на кнопку «Enter», если в строке не только пробелы, приложение отправляет данные сообщения на сервер. Если сервер успешно сохранил сообщение в базе данных, то он отправляет ответ, что сообщение успешно отправлено и оно появляется в чате. В противном случае, если сообщение не сохранено в базе данных, высвечивается уведомление, что сообщение не отправлено. Также в строке отправки сообщений можно копировать и вставлять текст из буфера обмена при помощи стандартных

сочетаний клавиш «Ctrl+C» и «Ctrl+V». При вставке текста из буфера обмена, если в тексте есть переход на новую строку, то они все заменяются на пробел.

Код для сохранения сообщения в базу данных на стороне сервера представлен ниже:

```
public string SendMessageFriend(string message, string sender, string
recipient, DateTime dateTime)
{
    string answer;
    SqlConnection connection = new SqlConnection(@"Data
Source=DENNY\SERVER;Initial Catalog=GameChat;Integrated Security=True");
    connection.Open();
    string query = $"insert into FriendMessages(sender_id, recipient_id,
message, message_date, status) " + $"values((select user_id from Users where
user_name = '{sender}'), (select user_id from Users where user_name =
'{recipient}'), " + $"'{message}', '{dateTime}', 'не прочитано')";
    SqlCommand cmd = new SqlCommand(query, connection);
    if (cmd.ExecuteNonQuery() == 1)
    {
        answer = "сообщение отправлено";
        for (int i = 0; i < users.Count; i++) if (users[i].user_name == recipient)
users[i].operationContext.GetCallbackChannel<IService2CallBack>().SendMessage
FriendCallBack(message, sender, dateTime);
    }
    else answer = "ошибка";
    connection.Close();
    return answer;
}
```

После успешного сохранения сообщения в базе данных, если в списке пользователей, у которых открыт мессенджер, есть пользователь, которому отправляют сообщение, то сервер отправляет этому пользователю сообщение на форму, если она открыта. В противном случае, если чат не открыт, то появляется уведомление о новом сообщении. На рисунке 9 представлено как выглядит уведомление о новом сообщении.

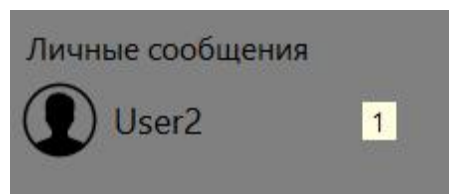


Рисунок 9 — Уведомление о новом сообщении

4.6 Реализация комнат

Комнаты разделены по играм. Перед тем чтобы создать комнату, нужно выбрать игру, для которой предназначена комната. Для этого нужно нажать на вкладку «Комнаты по играм». При открытии данной формы клиентское

приложение запрашивает список всех имеющихся в базе данных игр. После по названию каждой игры запрашивает его изображение:

```
public List<string> getGames()
{
    List<string> answer = new List<string>();
    SqlConnection connection = new SqlConnection(@"Data
Source=DENNY\SERVER;Initial Catalog=GameChat;Integrated Security=True");
    connection.Open();
    SqlDataAdapter adapter = new SqlDataAdapter();
    DataTable table = new DataTable();
    string query = $"select game_name from Games;";
    SqlCommand cmd = new SqlCommand(query, connection);
    adapter.SelectCommand = cmd;
    adapter.Fill(table);
    if (table.Rows.Count > 0)
    {
        for (int i = 0; i < table.Rows.Count; i++)
            answer.Add(table.Rows[i][0].ToString());
    }
    connection.Close();
    return answer;
}

public byte[] getImageGame(string game_name)
{
    SqlConnection connection = new SqlConnection(@"Data
Source=DENNY\SERVER;Initial Catalog=GameChat;Integrated Security=True");
    connection.Open();
    SqlDataAdapter adapter = new SqlDataAdapter();
    DataTable table = new DataTable();
    string query = $"select game_image from Games where game_name =
'{game_name}'";
    SqlCommand cmd = new SqlCommand(query, connection);
    var answer = cmd.ExecuteScalar();
    connection.Close();
    return answer as byte[];
}
```

На рисунке 10 представлен интерфейс данной вкладки.

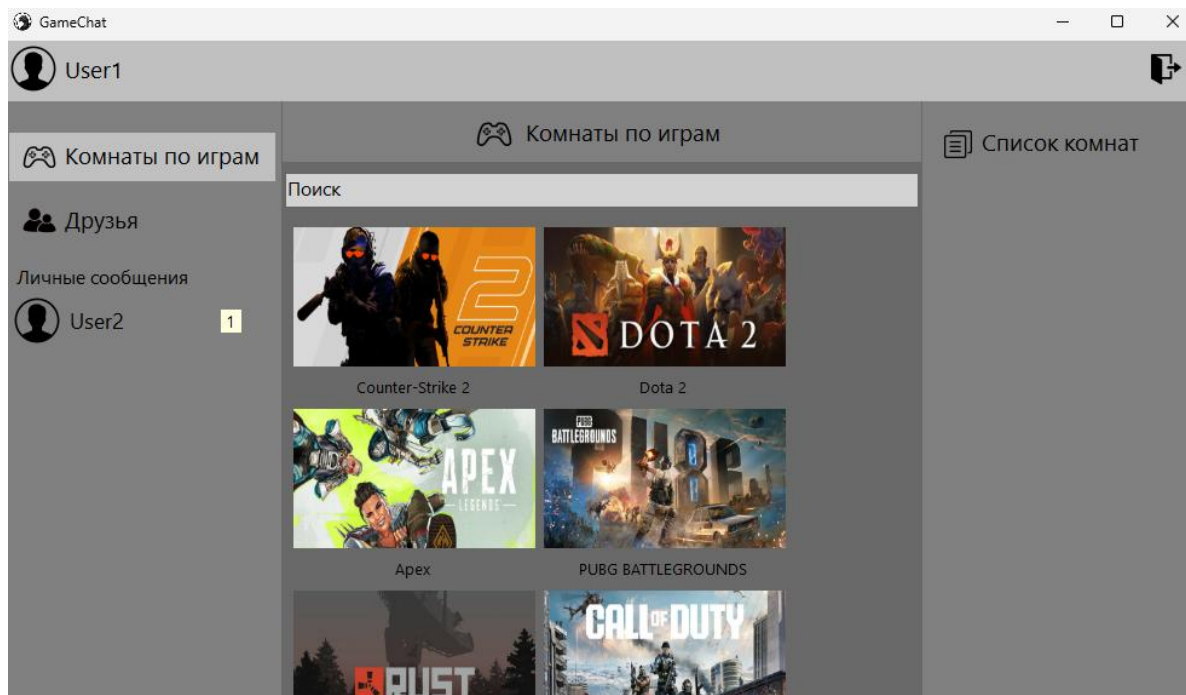


Рисунок 10 — Интерфейс вкладки «Комнаты по играм»

На данной форме расположены игры, в которых пользователи могут создавать комнаты для общения и обмена информацией по данной игре. Также сверху есть поле для быстрого поиска нужной игры. При нажатии на игру открывается форма, в которой отображаются все комнаты по этой игре. При открытии формы мессенджер запрашивает у сервера данные о всех имеющихся комнатах по выбранной игре:

```
public List<string[]> getRooms(string game_user, string game_or_user)
{
    List<string[]> answer = new List<string[]>();
    SqlConnection connection = new SqlConnection(@"Data
Source=DENNY\SERVER;Initial Catalog=GameChat;Integrated Security=True");
    connection.Open();
    SqlDataAdapter adapter = new SqlDataAdapter();
    DataTable table = new DataTable();
    string query = "";
    if (game_or_user == "game") query = $"select room_id, room_name,
password, members_current, members_max from Rooms where game_id = (select
game_id from Games where game_name = '{game_user}')";
    else if (game_or_user == "user") query = $"select Rooms.room_id,
room_name, password, members_current, members_max, game_name from Rooms "
+ $"join RoomMembers on Rooms.room_id = RoomMembers.room_id " + $"join
Games on Rooms.game_id = Games.game_id " + $"where user_id = (select user_id
from Users where user_name = '{game_user}') and status != 'забанен' " + $"order
by game_name asc";
    SqlCommand cmd = new SqlCommand(query, connection);
    adapter.SelectCommand = cmd;
    adapter.Fill(table);
    if (table.Rows.Count > 0)
```

```

{
for (int i = 0; i < table.Rows.Count; i++)
{
string[] row = new string[4];
if (game_or_user == "user") row = new string[5];
row[0] = table.Rows[i][0].ToString();
row[1] = table.Rows[i][1].ToString();
row[2] = table.Rows[i][2].ToString();
if (table.Rows[i][4].ToString() == "") row[3] = "null";
else row[3] = $"{table.Rows[i][3]}/{table.Rows[i][4]}";
if (game_or_user == "user") row[4] = table.Rows[i][5].ToString();
answer.Add(row);
}
}

```

На рисунке 11 представлен интерфейс список комнат в игре Dota 2.

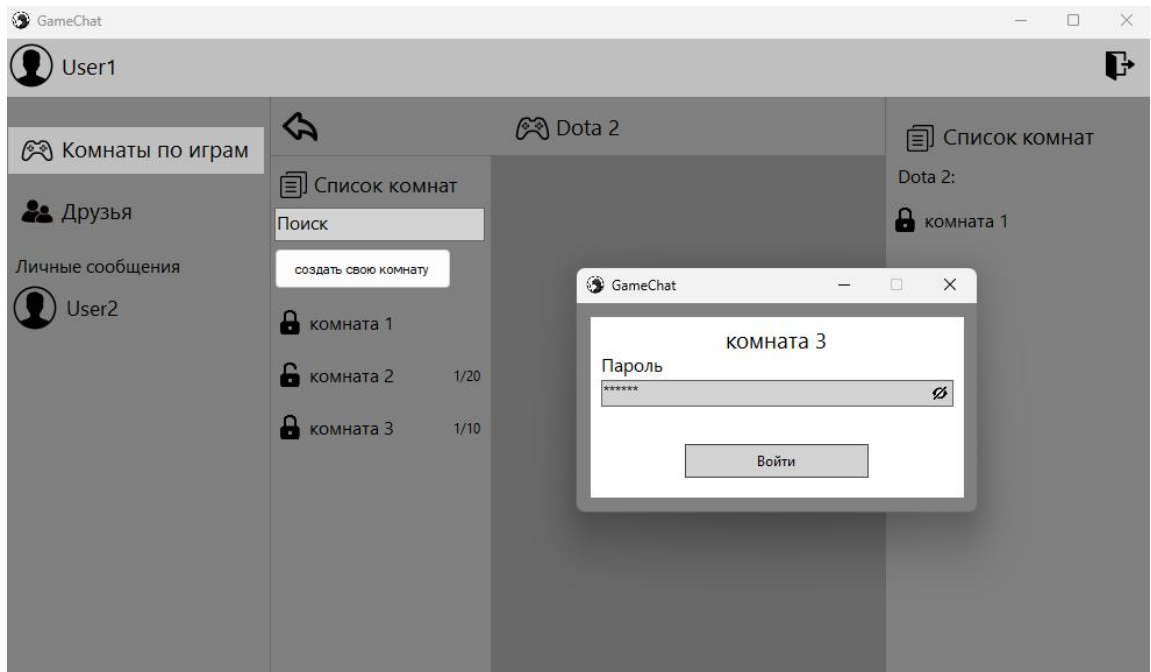


Рисунок 11 — Интерфейс «Список комнат»

Сверху формы отображается название игры в которой находятся комнаты. Слева показаны сами комнаты, а также строка для поиска комнаты по названию и кнопка для создания своей комнаты. При нажатии на кнопку открывается форма для создания комнаты. На рисунке 12 показано как выглядит форма для создания комнаты.

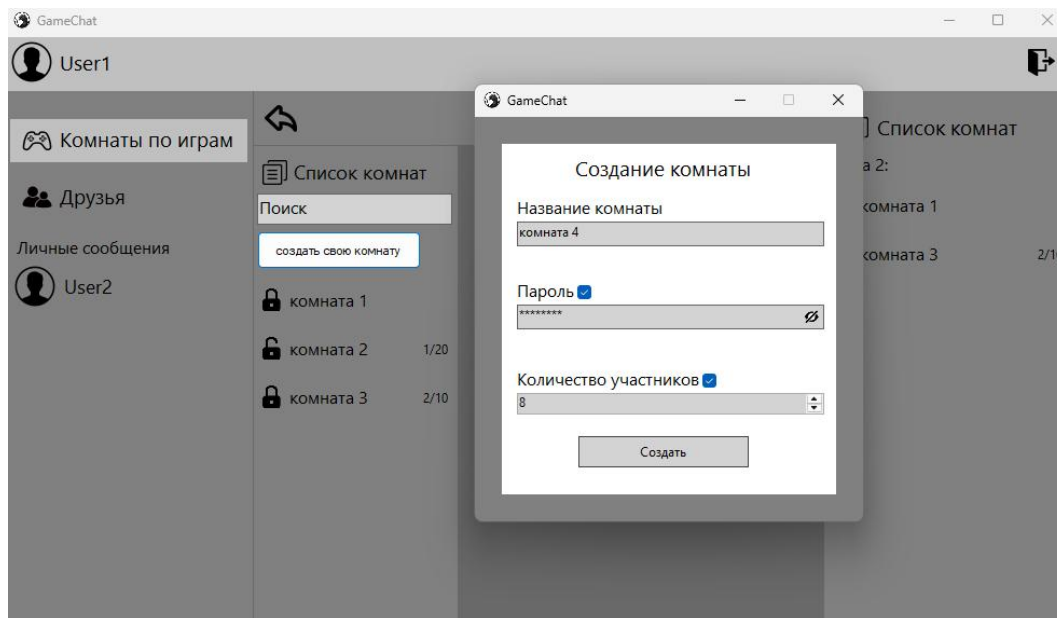


Рисунок 12 — Создание комнаты в игре «Dota 2»

При создании комнаты обязательно нужно ввести название комнаты. Название комнаты не может быть меньше четырех символов, иначе появится сообщение, предупреждающее о слишком коротком названии. Также пароль не может быть меньше 6 символов:

```
private void buttonCreate_Click(object sender, EventArgs e)
{
    if (textBoxRoomName.Text.Length < 4)
    {
        labelErrorRN.Text = "Имя комнаты не может быть меньше 4 символов";
        labelErrorRN.Visible = true;
    }
    else if (checkBoxPassword.Checked && textBoxPassword.Text.Length < 6)
    {
        labelErrorPass.Text = "Пароль не может быть меньше 6 символов";
        labelErrorPass.Visible = true;
    }
    else
    {
        string password = textBoxPassword.Text;
        int members = (int)numUDMembers.Value;
        if (!checkBoxPassword.Checked) password = "null";
        if (!checkBoxMembers.Checked) members = 0;
        client = new ServiceClient();
        string answer = client.createRoom(user_name, game,
        textBoxRoomName.Text, password, members);
        if (answer == "Имя комнаты уже существует в этой игре")
        {
            labelErrorRN.Text = answer;
            labelErrorRN.Visible = true;
        }
    }
}
```

```

}
else if (answer == "ошибка") MessageBox.Show(answer);
else if (answer == "комната создана")
{
roomsByGame.roomsDisplay();
roomsByGame.openRoom(textBoxRoomName.Text);
roomsByGame.mainScreen.roomsDisplay();
Close();
}
}
}
}

```

Пароль и количество участников необязательны, их можно настроить по желанию, изначально они отключены. Если в комнате имеется пароль, то слева от названия комнаты будет изображен закрытый замок, иначе, если пароля нет, то замок будет открытым. Также и с количеством участников, если этот параметр включен, то справа будет отображаться текущее и максимальное количество участников, в противном случае там будет пусто. При нажатии на кнопку «Создать» мессенджер передаёт данные о комнате серверу, который в свою очередь записывает в базу данных. Если сервер не смог сохранить комнату в базу данных или комната с таким названием в этой игре уже существует, то высветится сообщение об ошибке. В противном случае комната сохранится в базу данных и отобразится в мессенджере:

```

public string createRoom(string user_creator, string game, string room_name,
string password, int members_max)
{
string answer;
SqlConnection connection = new SqlConnection(@"Data
Source=DENNY\SERVER;Initial Catalog=GameChat;Integrated Security=True");
connection.Open();
SqlDataAdapter adapter = new SqlDataAdapter();
DataTable table = new DataTable();
string query = $"select room_name from Rooms where game_id = (select
game_id from Games where game_name = '{game}') and room_name =
'{room_name}';";
SqlCommand cmd = new SqlCommand(query, connection);
adapter.SelectCommand = cmd;
adapter.Fill(table);
if (table.Rows.Count < 1)
{
adapter = new SqlDataAdapter();
if (members_max > 0) query = $"insert into Rooms(game_id, room_name,
password, members_current, members_max) " + $"values((select game_id from
Games where game_name = '{game}'), '{room_name}', '{password}', {1},
{members_max});";
else query = $"insert into Rooms(game_id, room_name, password,
members_current, members_max) " + $"values((select game_id from Games where

```

```

game_name = '{game}', '{room_name}', '{password}', null, null);";
cmd = new SqlCommand(query, connection);
adapter.SelectCommand = cmd;
if (cmd.ExecuteNonQuery() == 1)
{
    adapter = new SqlDataAdapter();
    query = $"insert into RoomMembers(user_id, room_id, status) " +
    $"values((select user_id from Users where user_name = '{user_creator}'), " +
    $"(select room_id from Rooms where room_name = '{room_name}' and game_id = " +
    $(select game_id from Games where game_name = '{game}')), 'создатель')";
    cmd = new SqlCommand(query, connection);
    adapter.SelectCommand = cmd;
    cmd.ExecuteNonQuery();
    answer = "комната создана";
}
else answer = "ошибка";
}
else answer = "Имя комнаты уже существует в этой игре";
connection.Close();
return answer;
}

```

При открытии комнаты приложение запрашивает у сервера все сообщения в комнате и отображает их в чате:

```

public List<string[]> getMessagesInRoom(int room_id)
{
    List<string[]> answer = new List<string[]>();
    SqlConnection connection = new SqlConnection(@"Data
Source=DENNY\SERVER;Initial Catalog=GameChat;Integrated Security=True");
    connection.Open();
    SqlDataAdapter adapter = new SqlDataAdapter();
    DataTable table = new DataTable();
    string query = $"select Users.user_name, message, message_date from
RoomMessages " + $"join Users on RoomMessages.sender_id = Users.user_id
where room_id = {room_id}";
    SqlCommand cmd = new SqlCommand(query, connection);
    adapter.SelectCommand = cmd;
    adapter.Fill(table);
    if (table.Rows.Count > 0)
    {
        for (int i = 0; i < table.Rows.Count; i++)
        {
            string[] row = new string[3];
            row[0] = table.Rows[i][0].ToString();
            row[1] = table.Rows[i][1].ToString();
            row[2] = table.Rows[i][2].ToString();
            answer.Add(row);
        }
    }
}

```



```

}
}
connection.Close();
return answer;
}

```

На рисунке 13 изображен интерфейс чата в комнате.

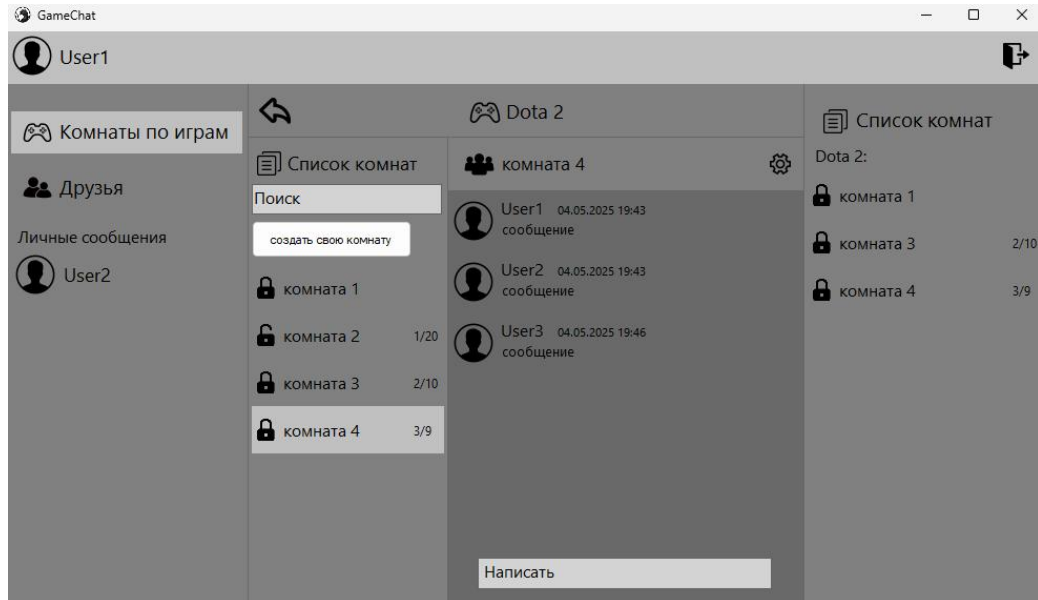


Рисунок 13 — Интерфейс чата в комнате

Сверху отображается название комнаты и значок шестеренки для настройки комнаты. Посередине отображаются сообщения пользователей, снизу строка для отправки сообщения. Отправка сообщений в комнате работает по тому же принципу, что и в личных сообщениях, только вместо отправки сообщения получателю оно хранится в комнате и отображается у всех участников комнаты:

```

public string SendMessageInRoom(int room_id, string sender_name, string
message, DateTime dateTime)
{
    string answer = "";
    SqlConnection connection = new SqlConnection(@"Data
Source=DENNY\SERVER;Initial Catalog=GameChat;Integrated Security=True");
    connection.Open();
    string query = $"insert into RoomMessages(room_id, sender_id, message,
message_date) " + $"values({room_id}, (select user_id from Users where user_name
='{sender_name}'), '{message}', '{dateTime}')";
    SqlCommand cmd = new SqlCommand(query, connection);
    if (cmd.ExecuteNonQuery() == 1)
    {
        answer = "сообщение отправлено";
        SqlDataAdapter adapter = new SqlDataAdapter();
        DataTable table = new DataTable();
        query = $"select Users.user_name from RoomMembers " + $"join Users on

```

```

RoomMembers.user_id = Users.user_id " + $"where RoomMembers.room_id =
{room_id} and RoomMembers.status != 'забанен' and Users.user_name !=
'{sender_name}'";
cmd = new SqlCommand(query, connection);
adapter.SelectCommand = cmd;
adapter.Fill(table);
if (table.Rows.Count > 0)
{
for (int i = 0; i < users.Count; i++)
{
for (int j = 0; j < table.Rows.Count; j++)
{
if (users[i].user_name == table.Rows[j][0].ToString())
{
users[i].operationContext.GetCallbackChannel<IService2CallBack>().Send
MessageInRoomCallBack(room_id, sender_name, message, dateTime);
break;
}
}
}
}
}
else answer = "ошибка";
connection.Close();
return answer;
}

```

При нажатии на шестеренку открывается форма для настройки комнаты, в которой можно сменить пароль или удалить комнату. На рисунке 14 представлена форма для настройки комнаты.

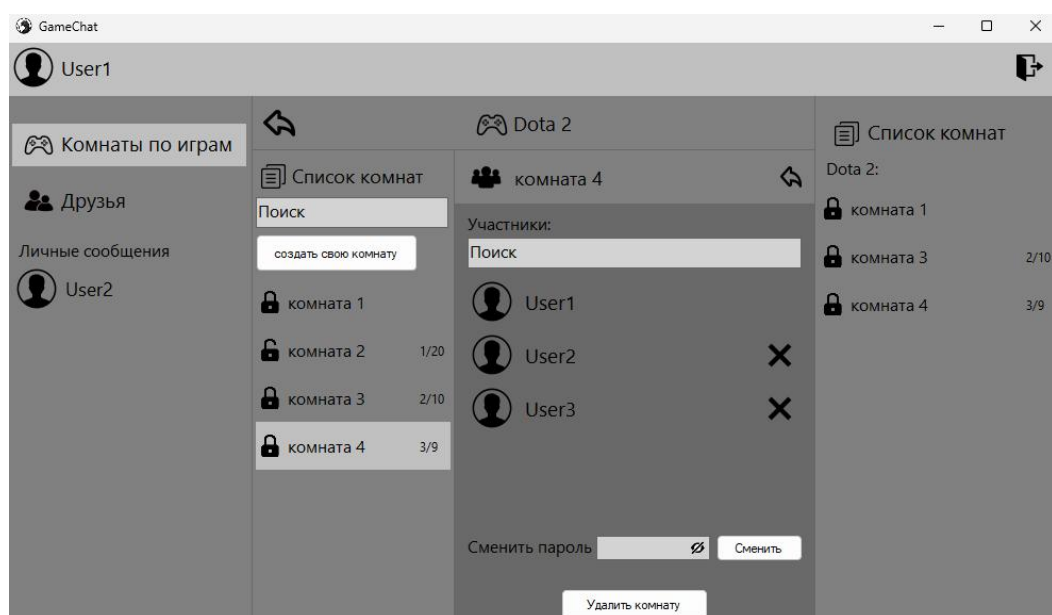


Рисунок 14 — Форма для настройки комнаты

Также на форме отображается список участников комнаты и имеется строка для быстрого поиска пользователей. Нажав на крестик напротив имени пользователя он будет выгнан и не сможет зайти в комнату:

```
public string ban(int room_id, string user_name, string game)
{
    string answer;
    SqlConnection connection = new SqlConnection(@"Data
Source=DENNY\SERVER;Initial Catalog=GameChat;Integrated Security=True");
    connection.Open();
    string query = $"update RoomMembers set status = 'забанен' " + $"where
room_id = {room_id} " + $"and user_id = (select user_id from Users where
user_name = '{user_name}')";
    query += $"update Rooms set members_current =
members_current - 1 where room_id = {room_id}";
    SqlCommand cmd = new SqlCommand(query, connection);
    if (cmd.ExecuteNonQuery() == 2) answer = "пользователь забанен";
    else answer = "ошибка";
    try
    {
        for (int i = 0; i < users.Count; i++) if (users[i].user_name == user_name)
            users[i].operationContext.GetCallbackChannel<IService2CallBack>().banCallBack
            (room_id, game);
    }
    catch { }
    connection.Close();
    return answer;
}
```

Код при помощи которого выполняются остальные действия над комнатой:

```
public string roomSetting(string operation, string operation_object)
{
    string answer = "";
    SqlConnection connection = new SqlConnection(@"Data
Source=DENNY\SERVER;Initial Catalog=GameChat;Integrated Security=True");
    connection.Open();
    string query;
    SqlCommand cmd;
    switch (operation)
    {
        case "сменить пароль":
            query = $"update Rooms set password = '{operation_object.Split(';')[1]}'
where room_id = {operation_object.Split(';')[0]}";
            cmd = new SqlCommand(query, connection);
            if (cmd.ExecuteNonQuery() == 1) answer = "пароль изменён";
            else answer = "ошибка";
            break;
        case "удалить комнату":
```

```

        query = $"delete from RoomMembers where room_id =
{operation_object};\r\n" + $"delete from Rooms where room_id =
{operation_object}";
        cmd = new SqlCommand(query, connection);
        if (cmd.ExecuteNonQuery() == 2) answer = "комната удалена";
        else answer = "ошибка";
        break;
        case "разбанить участника":
            query = $"delete from RoomMembers where room_id =
{operation_object.Split(';')[0]} " + $"and user_id = (select user_id from Users where
user_name = '{operation_object.Split(';')[1]}')";
            cmd = new SqlCommand(query, connection);
            if (cmd.ExecuteNonQuery() == 1) answer = "пользователь разбанен";
            else answer = "ошибка";
            break;
        }
        connection.Close();
        return answer;
    }

```

5 ЭКОНОМИЧЕСКАЯ ЧАСТЬ

Экономическая эффективность – процесс, подразумевающий анализ ресурсов, которые потребуются для реализации проекта.

В экономической части необходимо определить величину денежных средств, необходимую численность персонала, срок окупаемости и величину рентабельности собственных средств и продаж.

5.1 Экономические затраты на разработку проекта

Перечень расходов и доходов на разработку продукта состоит из:

- оплаты труда разработчика;
- отчисления во внебюджетные фонды;
- оплаты электроэнергии;
- оплаты машинного времени;
- прочие затраты.

5.1.1 Расчет оплаты труда разработчика. Расходы на оплату труда разработчика программы (ЗПраз) определяется путем умножения трудоемкости создания программы на среднюю часовую заработную плату специалиста. Формула расходов на оплату труда разработчика проекта:

$$\text{ЗП}_{\text{раз}} = T * \text{СЧ}_{\text{раз}} \quad (1)$$

где T – трудоемкость разработки проекта;

$\text{СЧ}_{\text{раз}}$ – средняя часовая оплата труда разработчика, рублей в час.

$$\text{СЧ}_{\text{раз}} = \text{П}_{\text{раз}} / \text{Ф}_{\text{рв}} \quad (2)$$

где $\text{П}_{\text{раз}}$ – заработная плата программиста (по информации службы занятости средняя заработная плата программиста составляет 111 860 руб.);

$\text{Ф}_{\text{рв}}$ – месячный фонд рабочего времени, при 40-часовой рабочей неделе он будет равен: $\text{Ф}_{\text{рв}} = 176 \text{ часов} = ((40 \text{ часов} / 5 \text{ дней}) * 22 \text{ дня})$;

Подставляя в формулу данные значения, получим значения $\text{СЧ}_{\text{раз}}$ и $\text{ЗП}_{\text{раз}}$:

$$\text{СЧ}_{\text{раз}} = 111\,860 / 176 = 635,5 \text{ руб./час}$$

Общая трудоемкость разработки программы T составляет 528 человеко-часов (за 3 месяца работы с месячным фондом рабочего времени 176 часов)

$$\text{ЗП}_{\text{раз}} = 528 * 635,5 = 335544 \text{ руб.}$$

5.1.2 Расчет отчислений во внебюджетные фонды. Расчет отчислений во внебюджетные фонды составляет 30% от затрат на оплату труда разработчика:

$$\text{Соц.Отч.} = \text{ЗП}_{\text{раз}} * 0,3 \quad (3)$$

$$\text{Соц.Отч.} = 335544 * 0,3 = 100663 \text{ руб.}$$

Таким образом, расходы на оплату труда разработчика программы составляет 436207 рублей.

5.1.3 Расчет затрат на оплату машинного времени. Затраты на оплату машинного времени при разработке программы рассчитывается по следующей формуле:

$$\text{З}_{\text{мв}} = C * (t_{\text{н}} + t_{\text{отл}}) \quad (4)$$

где C – цена машино-часов;

t_n – количество машино-часов разработки программы;

$tot_{пл}$ – затраты на отладку проекта.

Рассчитываемая цена машино-часа:

$$C = (Z_a + Z_{mv} + Z_{tr} + Z_{pr}) / T_{пк} \text{ (руб./год)} \quad (5)$$

где Z_a – затраты на амортизацию, годовые издержки на амортизацию, рублей в год;

Z_{mv} – годовые издержки на вспомогательные материалы, рублей в год;

Z_{tr} – затраты на текущий ремонт компьютера, рублей в год;

Z_{pr} – годовые издержки на прочие и накладные расходы, рублей в год;

$T_{пк}$ – действительный годовой фонд времени ЭВМ, часов в год.

Далее, рассчитываются годовые издержки на амортизацию по формуле:

$$Z_a = C_{бал} * N_a / 100 \quad (6)$$

где $C_{бал}$ – балансовая стоимость компьютера, руб./шт.;

N_a – норма амортизации в процентах.

Балансовая стоимость компьютера определяется по формуле:

$$C_{бал} = C_{пер} + Z_{пр} \quad (7)$$

где $C_{пер}$ – рыночная стоимость компьютера, в рублях;

$Z_{пр}$ – прочие затраты (доставка, установка от 8 до 12% от стоимости компьютера)

Возьмем $Z_{пр} = 0\%$ от рыночной стоимости компьютера. Определение рыночной стоимости компьютера отображено в таблице 1.

Таблица 1 – Рыночная стоимость компьютера

Оборудование	Цена
Ноутбук	40000 руб.
Мышь	1600 руб.
Итого:	41600 руб.

Согласно данной таблице, рыночная стоимость компьютера составляет 41600 рублей.

Срок службы компьютера составляет 5 лет, отсюда норма амортизации составит 20%. Рассчитаем затраты на доставку и установку:

$$Z_{пр} = 41600 \text{ руб.} * 0 = 0 \text{ руб.}$$

Рассчитаем балансовую стоимость компьютера:

$$C_{бал} = 41600 \text{ руб.} + 0 = 41600 \text{ руб.}$$

По формуле (9) рассчитываются годовые издержки на амортизацию:

$$Z_a = 41600 * 20 / 100 = 8320 \text{ руб.}$$

Рассчитываем годовые издержки на вспомогательные материалы:

$$Z_{mv} = C_{бал} * 0,02 = 41600 * 0,02 = 832 \text{ руб.} \quad (8)$$

Рассчитываем затраты на текущий ремонт компьютера:

$$Z_{tr} = C_{бал} * 0,05 = 41600 * 0,05 = 2080 \text{ руб.} \quad (9)$$

Рассчитываем годовые издержки на прочие и накладные расходы:

$$Z_{pr} = C_{бал} * 0,07 = 41600 * 0,07 = 2912 \text{ руб.} \quad (10)$$

Рассчитываем действительный годовой фонд времени ЭВМ по следующей формуле:

$$T_{пк} = N_m * N_d * N_{ч} \quad (11)$$

где N_m – количество месяцев в году (12 месяцев);
 N_d – количество рабочих дней в месяце (22 недели);
 $N_{ч}$ – средняя продолжительность рабочего дня (8 часов).

$$T_{пк} = 12 * 22 * 8 = 2112 \text{ часов/год.}$$

Действительный годовой фонд времени ЭВМ равен 2112 часов. По формуле (5) рассчитываем цену одного машино-часа:

$$C = (8320 + 832 + 2080 + 2912) / 2112 = 7 \text{ руб.}$$

Рассчитываем по формуле (4) затраты на оплату машинного времени при написании и отладке программы:

$$З_{мв} = 7 * (130 + 160) = 2030 \text{ руб.}$$

5.1.4 Расчет затрат на электроэнергию. Расчет затрат на электроэнергию производится по формуле:

$$C_э = З_э * P * (t_n + t_{отл} + t_d) \quad (12)$$

где $З_э$ – стоимость 1кВтч электроэнергии (4,64 руб. / кВтч);

P – мощность, потребляемая компьютером (около 450 Вт);

t_n – затраты на машино-часов на проектирование системы;

$t_{отл}$ – затраты на отладку;

t_d – затраты на подготовку документации.

$$C_э = 4,64 * 0,45 * (130 + 160 + 90) = 793,44 \text{ руб.}$$

Таким образом затраты на электроэнергию при разработке программы составят 793,44 рублей.

5.1.5 Калькуляция системной стоимости. Системная стоимость – эксплуатационные расходы на разработку проекта.

Рассчитываем прочие затраты при разработке программы (они составляют от 5 до 9% от суммы остальных затрат):

$$З_{п} = З_{все} * 0,09,$$

$$З_{все} = З_{Праз} + Соц.Отч. + З_{мв} + C_э,$$

$$З_{все} = 335544 + 100663 + 2030 + 793,44 = 439030,44 \text{ руб.}, \quad (13)$$

$$З_{п} = 439030,44 * 0,09 = 39513 \text{ руб.}$$

Рассчитаем смету затрат на разработку программного продукта по формуле:

$$З_{общ} = З_{Праз} + Соц.Отч. + З_{мв} + C_э + З_{п} \quad (14)$$

$$З_{общ} = 335544 + 100663 + 2030 + 793,44 + 39513 = 478543 \text{ руб.}$$

Сметная стоимость проекта составит 478543 руб. Для наилучшей наглядности составлена таблица 2, в которой сведены полученные сметы затрат.

Таблица 2 – Экономические затраты на разработку системы

Статьи затрат	Индекс	Сумма, руб	Структура, %
Заработная плата разработчика	$З_{Праз}$	335544	70,12
Отчисления во внебюджетные фонды	Соц.Отч	100663	21,04
Затраты на оплату	$З_{мв}$	2030	0,42

машинного времени			
Затраты на электроэнергию	$C_э$	793,44	0,17
Прочие затраты	$З_п$	39513	8,26
Итого:	$З_{общ}$	478543	100

Из таблицы 2 видно, что большая часть затрат при разработке программы приходится на заработную плату разработчику и выплату с этой зарплаты единого социального налога.

5.2 Капитальные затраты на внедрение программы

Рассчитываем затраты на приобретение оборудования и ПО для программы.

Необходимое оборудование:

Компьютер с необходимыми системными требованиями и периферийными устройствами – 41600 руб.

Операционная система Windows 10 – 4000 руб.

Рассчитаем затраты на приобретение необходимого технического и программного обеспечения функционирования системы по формуле:

$$C_{доп} = C_{об} + C_{пр} \quad (15)$$

где $C_{доп}$ – расходы на оборудование и ПО;

$C_{об}$ – стоимость оборудования;

$C_{пр}$ – стоимость программного обеспечения.

$$C_{доп} = 41600 + 4000 = 45600 \text{ руб.}$$

5.3 Экономический эффект от применения программы

5.3.1 Расчет дохода заказчика. Источником дохода от разработки программы для заказчика является продажа встроенных привилегий.

Таким образом, общий ежемесячный доход от программы вычисляется по формуле:

$$S_d = C_{ср} * N_{ср} \quad (16)$$

где $N_{ср}$ – среднее количество продаж встроенных привилегий;

$C_{ср}$ – стоимость привилегии.

$$S_d = 600 * 1000 = 600000 \text{ руб.}$$

5.3.2 Расчет прибыли заказчика. Прибыль рассчитывается как разница между доходом от программы и затратами на ее разработку

$$P = S_d - Z_{общ} \quad (17)$$

$$P = 600000 - 478543 = 121457 \text{ руб.}$$

5.3.3 Экономический эффект от применения программы. Период окупаемости внедрения программы рассчитывается по формуле:

$$T_{ок} = Z_{общ} / P \quad (18)$$

$$T_{ок} = 478543 / 121457 = 3,9 \text{ месяцев}$$

Таким образом, на основе проведенных расчетов можно сделать вывод, что программа является рентабельной, так как период ее окупаемости равен 3,9 месяцев.

ЗАКЛЮЧЕНИЕ

В результате выполнения дипломного проекта специалиста решены следующие задачи:

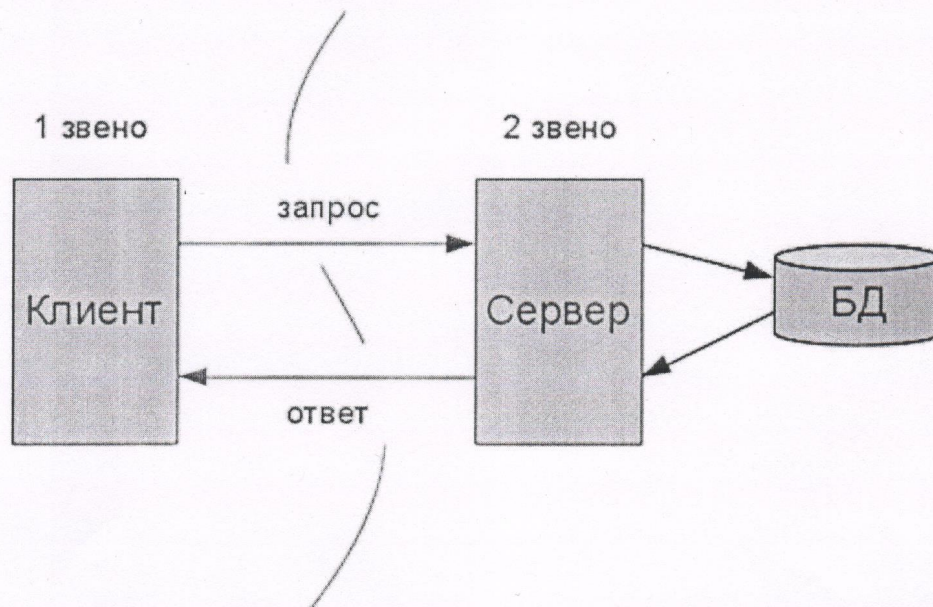
- анализ требований пользователей;
- проектирование архитектуры приложения;
- создание пользовательского интерфейса;
- разработка функционала;
- обеспечение безопасности данных.

В дальнейшем планируется:

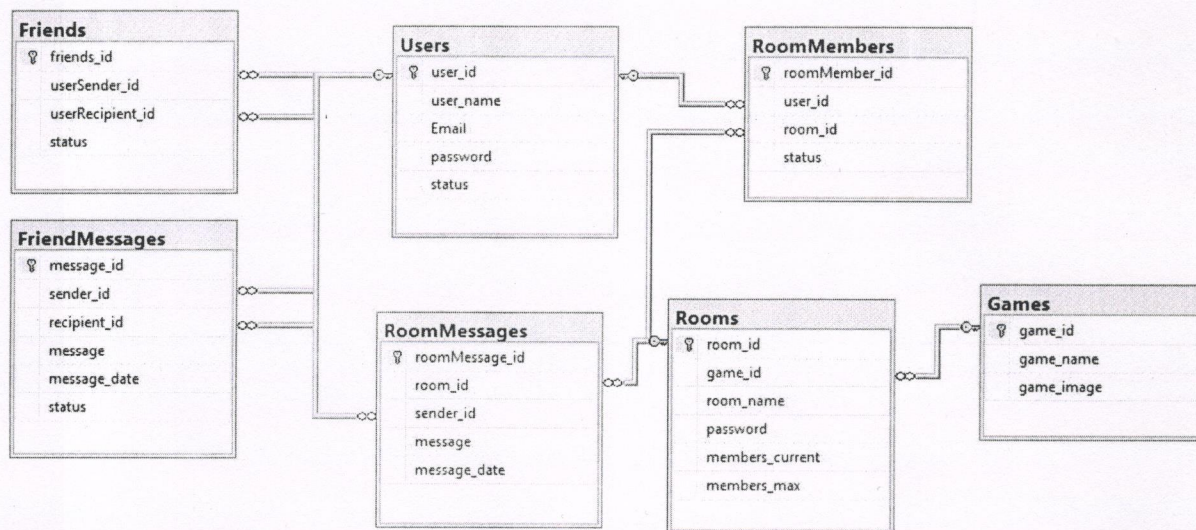
- разработать и внедрить дополнительные функции, такие как передача голосовых сообщений, видеозвонки, возможность обмена файлами и интеграция с игровыми платформами для улучшения взаимодействия пользователей;
- провести комплексное тестирование для выявления и устранения ошибок, а также для оптимизации работы приложения;
- организовать сбор отзывов и предложений от пользователей для дальнейшего улучшения функционала и удобства использования мессенджера.

СПИСОК ЛИТЕРАТУРЫ

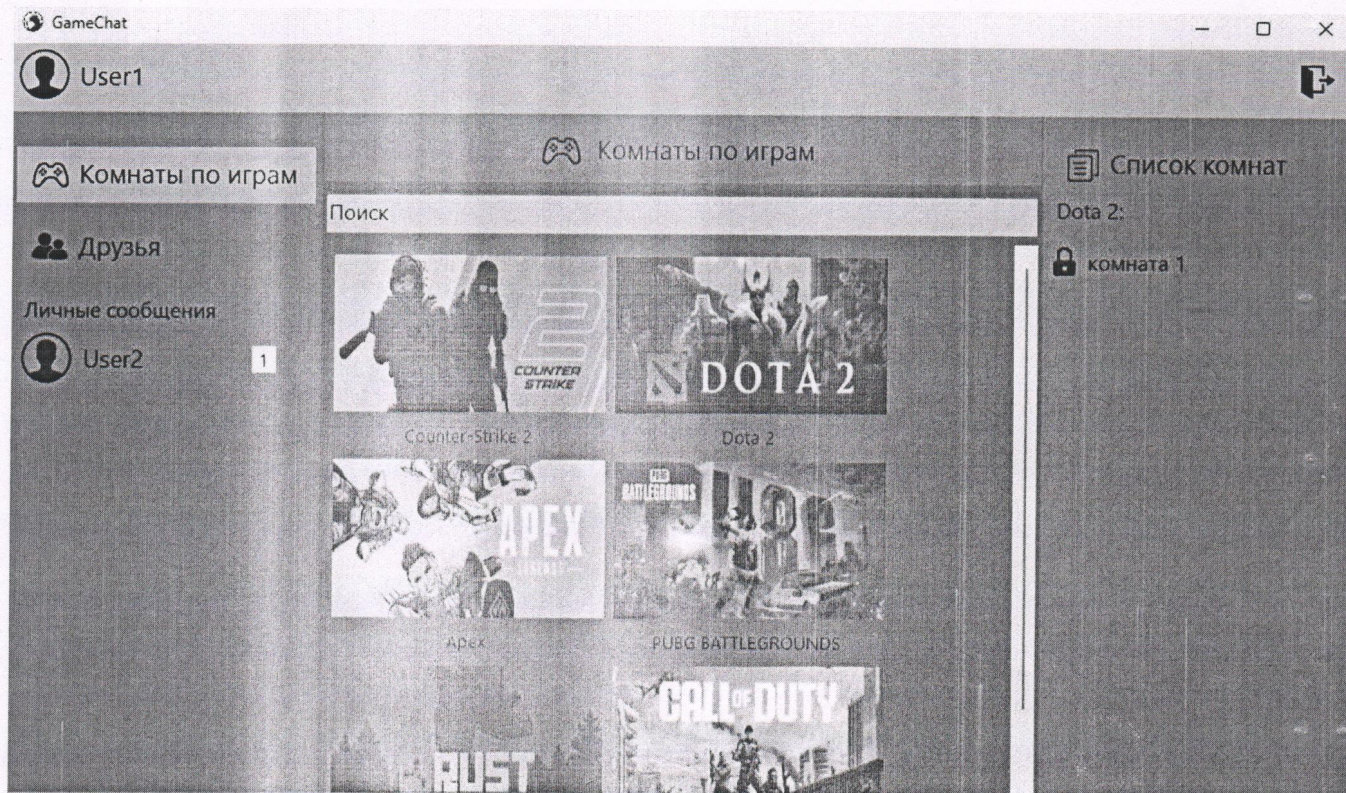
- 1) Ермолов Д. Создание клиент-серверных приложений на С# с использованием WCF. – СПб.: Питер, 2018. – 496 с.
- 2) Поляков А. SQL Server 2019. Проектирование и администрирование баз данных. – М.: ДМК-Пресс, 2020. – 704 с.
- 3) Гриффитс И. Программируем на С# 8.0. Разработка приложений. – СПб.: Питер, 2021. – 944 с.
- 4) Смирнов К. Архитектура и проектирование программного обеспечения на С#. – М.: Эксмо, 2021. – 544 с.
- 5) Снеговский М. Построение распределенных систем на основе WCF. – СПб.: Питер, 2017. – 464 с.
- 6) Холленд Р. Microsoft SQL Server 2019. Полное руководство. – М.: ДМК Пресс, 2020. – 1400 с.
- 7) Виейра Р. Разработка приложений Windows Forms на С# для профессионалов. – М.: Вильямс, 2018. – 864 с.



					ДП 09.02.07. 10. Д. 01 СХ			
Изм.	Лист	№ докум.	Подп.	Дата	Схема клиент-серверной архитектуры	Лит.	Лист	Листов
Разраб.		Костюк И.В.	<i>[Signature]</i>	11.06.25		У	1	1
Пров.		Дегтярёв А.Д.	<i>[Signature]</i>	11.06.25		КТИ (филиал) ВолгГТУ		
Н.контр.		Ромашенко А.И.	<i>[Signature]</i>	11.06.25				
Утв.		Харитонов И.М.	<i>[Signature]</i>	11.06.25				



					ДП 09.02.07. 10. Д. 02 СХ						
Изм.	Лист	№ докум.	Подп.	Дата	ER-диаграмма базы данных			Лит.	Лист	Листов	
Разраб.		Костюк И.В.		11.06.25				У	1	1	
Пров.		Дегтярёв А.Д.		11.06.25							
Н.контр.		Ромашенко А.И.		11.06.25				КТИ (филиал) ВолгГТУ			
Утв.		Харитонов И.М.		11.06.25							



					ДП 09.02.07. 10. Д. 03 СХ			
Изм.	Лист	№ докум.	Подп.	Дата	Интерфейс главной формы	Лит.	Лист	Листов
Разраб.		Костюк И.В.		11.06.25		У	1	1
Пров.		Дегтярёв А.Д.		11.06.25				
Н.контр.		Ромашенко А.И.		11.06.25		КТИ (филиал) ВолГТУ		
Утв.		Харитонов И.М.		11.06.25				